High-Frequency Trading Order Book Benchmarking in C++20

Anthony Nguyen

Supervisor: Steve Pettifer

BSc (Hons) Computer Science

Department of Computer Science

UNIVERSITY OF MANCHESTER

April 23, 2026

Word Count: 14,865 (Verified via texcount)

Contents

Abstract **viii**

1 Introduction **1**
1.1 Context and Motivation 1
1.2 Problem Statement and Research Gap 1
1.3 Aim and Objectives 2
1.4 Research Hypotheses 2
1.5 Scope and Boundaries 3
1.6 Contributions Overview 3
1.7 Dissertation Structure 3

2 Background **4**
2.1 High-Frequency Trading and the Limit Order Book 4
2.2 Hardware Architecture 5
2.2.1 Cache Hierarchy and Memory Access 6
2.2.2 Branch Prediction 7
2.3 Data Structures for Order Books 7
2.3.1 Tree-Based Implementations 8
2.3.2 Flat Array Structures 8
2.3.3 Memory Pooling 9
2.4 Lock-Free Concurrent Queues 11
2.4.1 Exchange Architecture and Queue Modelling 11
2.4.2 SPSC and MPMC Implementations 11
2.4.3 Atomic Operations and Cache Coherence 13
2.4.4 The Standard Linux Network Stack 14
2.4.5 Kernel-Bypass Networking 16
2.5 Related Work 17

3 System Design **18**
3.1 Formal Requirements 18
3.1.1 Functional Requirements 19

3.1.2 Non-Functional Requirements 20
3.2 Design Goals and Constraints 20
3.3 High-Level Architecture 21
3.3.1 Architectural Pattern: Ports and Adapters 21
3.3.2 System Modules 21
3.4 The IOrderBook Interface and Polymorphism 23
3.5 Benchmarking Design: The Two-Mode Philosophy 23
3.6 Concurrency and Threading Strategy 24

4 Implementation **25**
4.1 Software Environment and Toolchain 25
4.2 Order Book Implementations 25
4.2.1 Standard Associative Map (Baseline) 26
4.2.2 Direct-Indexed Array (O(1) Access) 26
4.2.3 Hybrid "Hot-Cold" Vector-Map 27
4.2.4 Pool-Allocated Order Book (Slab Allocation) 27
4.3 The TCP Order Gateway 28
4.3.1 Protocol Specification and Parsing 28
4.3.2 The Loopback Simulation Model 30
4.4 High-Performance Concurrency 30
4.4.1 Wait-Free SPSC Queue 30
4.5 Low-Level Performance Utilities 30
4.5.1 Nanosecond Precision Timing 30
4.5.2 Thread-to-Core Affinity (CPU Pinning) 31
4.6 Automation and Data Collection 32

5 Testing and Evaluation **33**
5.1 Experimental Methodology 33
5.2 Functional Validation 35
5.2.1 Core Logic and Semantic Parity 35
5.2.2 System Pipeline and Integration 38
5.2.3 Scaling and Stress Testing 39
5.2.4 Gateway Simulation Boundaries and Validation 40
5.3 Throughput Results 40
5.4 Latency Analysis - Direct Mode 42
5.4.1 Architectural Mechanisms: Pool and Hybrid 46
5.4.2 Tail Latency Distribution and Determinism 49
5.5 Gateway Mode - End-to-End Decomposition 51
5.6 MPSC Thread Contention 54

5.7 Summary of Evaluation against Requirements 57
5.8 Threats to Validity 57

6 Conclusions **59**
6.1 Summary of Contributions 59
6.2 Answers to Research Hypotheses 59
6.3 Critical Reflection 60
6.3.1 Methodological Reflection 61
6.4 Limitations 61
6.5 Professional, Ethical, Social, and Legal Issues 62
6.6 Practical Recommendations 62
6.7 Future Work 63
6.8 Closing Statement 63

A Glossary **67**

B LockFreeQueue Implementation **70**

C ArrayOrderBook addOrder **72**

D Matching Engine Timestamp Decomposition **74**

E FIX Incremental Parser **76**

F PoolOrderBook Match Branching **78**

List of Figures

2.1 A stylized Limit Order Book demonstrating price-time priority grouping. The bid (buyers) and ask (sellers) are separated by the spread, establishing the mid-price. 5

2.2 CPU memory hierarchy illustrating the explicit size-latency tradeoff. Faster layers are closer to the execution core but exponentially smaller in capacity. 6

2.3 A chronological textbook view of a pipeline flush across discrete Clock Cycles (CC). When the branch (I_2) evaluates its condition at the Execute phase (CC4) and discovers a misprediction, all speculatively fetched incoming instructions (I_3, I_4) are forcefully discarded. The pipeline violently stalls until the correct target instructions (I_3') are fetched and advance through the queue. 8

2.4 Visual comparison of traditional heap allocation versus a Memory Pool. Dynamic allocations mathematically fragment across the heap as orders are rapidly added and deleted, forcing the CPU to fetch disjointed cache lines ("pointer chasing"). A Memory Pool allocates one massive contiguous block upfront, ensuring subsequent active object fetches naturally stream into the L1 hardware cache. 10

2.5 Simplified model of exchange message ingress. Packets arrive concurrently from multiple trading clients and are serialised by the Exchange NIC, which applies hardware timestamps and passes messages through an arbiter to enforce strict time-priority ordering. The resulting ordered stream is pushed into a Gateway Queue, from which the single Matching Engine thread consumes sequentially. 11

2.6 A lock-free Single-Producer Single-Consumer (SPSC) ring buffer. The producer actively writes to the Head pointer while the consumer asynchronously trails behind reading from the Tail pointer. Because the two threads concurrently modify distinct atomic variables, they operate entirely lock-free. So long as the underlying logic uses safe modulo arithmetic to wrap edges, the system remains infinitely scalable for isolated threads. 12

2.7 Multi-Producer Multi-Consumer (MPMC) queue contention model. Multiple concurrent producer threads (one per trading client) independently push order messages into the shared `moodycamel::ConcurrentQueue`. Atomic Head and Tail indices coordinate access without OS-level locking. A single consumer thread (the Matching Engine) dequeues messages sequentially, allowing the system to measure throughput degradation as producer count increases. 12
2.8 Structure of a FIX 4.2 "New Order Single" (MsgType=D) message. The protocol uses a human-readable tag-value encoding where fields are separated by the non-printable SOH character. While flexible, parsing this repeating sequence of ASCII characters induces significantly higher CPU overhead compared to binary protocols like SBE or FAST. 14
2.9 The standard Linux kernel network ingress pipeline. A packet must traverse multiple layers of abstraction—including DMA transfers, hardware interrupts, and TCP/IP stack logic—before reaching the application. Each stage, particularly the transition from kernel to user space via system calls, introduces non-deterministic delays that manifest as microsecond-scale jitter in HFT environments. 15
2.10 A comparison between the standard Linux networking stack and kernel-bypass technologies such as DPDK. While the standard stack requires the kernel to mediate packet delivery—introducing unpredictable context switches—kernel-bypass allows a user-space thread to poll the NIC hardware directly. This eliminates operating system overhead and provides the nanosecond-scale determinism required for HFT execution. 16

3.1 Data-Flow Overview: Pipeline stages from generation to telemetry. . . . 22
3.2 Hexagonal Architecture (Ports and Adapters): Clear separation between the matching domain and interchangeable adapters. 23

5.1 Mixed-scenario price distribution: the 55/30/15 split across near-spread, deep, and extreme price levels confirms the workload generator produces a heterogeneous order flow rather than collapsing into a single regime (NFR-1, NFR-9). 37
5.2 Mixed-scenario cancellation pressure: the 18% cancellation rate is verified against its theoretical configuration, validating benchmark integrity (NFR-1, NFR-9). 38
5.3 Full System Architecture: From Gateway Client to Matching Engine . . . 38
5.4 Direct-mode mean latency heatmap across scenarios and order book implementations. Array is the fastest implementation in all seven scenarios. 44

5.5 Direct-mode relative slowdown versus scenario winner (excluding vector). By removing the vector outlier, the performance nuances between the competitive non-vector implementations (Array, Hybrid, Pool, and Map) are preserved in the colormap scale. 45
5.6 Direct-mode standard-deviation heatmap (excluding vector). Lower values indicate tighter run-to-run stability, providing variance evidence alongside mean latency for NFR-6. 46
5.7 PoolOrderBook Matching Logic: Divergent Execution Paths 47
5.8 Cost decomposition of the `mixed` scenario (direct mode). The comparison highlights the close Hybrid-vs-map operation costs and the Pool's match-processing bottleneck. 49
5.9 Tail inflation map using P99-to-mean ratio in direct mode. Several map and vector cases exhibit elevated tail amplification relative to mean latency, indicating sensitivity to jitter and/or burst conditions. 51
5.10 End-to-End Telemetry Checkpoints. The pipeline is instrumented with four precise timestamps to isolate transport, queuing, and matching latencies. 52
5.11 Dense_full gateway decomposition (network, queue, engine) for non-vector implementations. The vector row is omitted in this comparative view because its dense_full queue and network outlier compresses the non-vector bars. 53
5.12 Gateway latency by scenario for non-vector implementations. Only scenarios with gateway measurements (`tight_spread`, `mixed`, and `dense_full`) are shown. The masking claim is supported by the decomposition in Table 5.8. 54
5.13 Throughput scaling under varying producer thread counts in MPSC mode. Identifies the contention point where the single consumer matching engine becomes the bottleneck. 55
5.14 Latency breakdown showing how queue wait times grow as more threads are added, while engine matching time stays the same. 56

List of Tables

3.1 Functional Requirements (FR) 19
3.2 Non-Functional Requirements (NFR) 20

5.1 Code Coverage Results for Core Order Book Logic 35
5.2 Mapping of FIX 4.2 Application Tags to Internal C++ Order Structures 39
5.3 Mean Throughput Across Implementations in Direct Mode (7 scenarios) and Gateway Mode (3 scenarios: `tight_spread`, `mixed`, `dense_full`) . . 42
5.4 Compact gateway decomposition evidence for primary non-vector books (derived from Table 5.8) 42
5.5 Representative direct-mode latency interval evidence (95% CI) 43
5.6 Mean Operation Latency in Direct Mode for `mixed` scenario (nanoseconds) 48
5.7 P99 Tail Latency in Direct Mode (nanoseconds) 50
5.8 End-to-End Latency Decomposition in Gateway Mode (nanoseconds), averaged across `tight_spread`, `mixed`, and `dense_full` 52
5.9 Compact MPSC contention evidence supporting the queue-bottleneck claim 57

Abstract

While algorithmic optimisation is a cornerstone of high-frequency trading (HFT), its practical impact is defined by the hardware-network stack. This dissertation presents a modular C++20 exchange infrastructure for benchmarking five order book implementations: array (direct-indexed), map (associative tree), vector (sorted), pool (slab-allocated), and hybrid (hot-cold tiered). These are evaluated in both direct and gateway modes. Three research hypotheses are evaluated: that cache-friendly indexing outperforms pointer-heavy alternatives (H1), that memory pooling reduces allocation costs without eliminating structural penalties (H2), and that system-level transport overhead masks engine-level gains (H3). To ensure scientific rigour, each claim is verified through Welch's t-tests with Bonferroni corrections and 95% confidence intervals ($n = 30$). In direct mode, the array implementation achieves a mean latency of 210 ns across scenarios, decisively outperforming all alternatives ($p < 0.0001$). However, in gateway mode, a quad-timestamp decomposition reveals that transport and queueing consume 88–96% of total latency, causing implementations to become statistically indistinguishable at the end-to-end level ($p = 0.21$). These results identify where engineering effort should transition from matching engine optimisation to transport-layer efficiency. All metrics are bounded to a pinned WSL2/AMD Ryzen environment.

Chapter 1

Introduction

1.1 Context and Motivation

In high-frequency trading (HFT), firms compete to execute orders at the lowest possible latency (Aldridge, 2013; Bilokon and Gunduz, 2023). In this environment, the order book serves as the fundamental state-machine on the critical path. The design of this component is a system-level problem where cache-coherence, branch-prediction, and network-stack overhead intersect to define performance (Gould et al., 2013; Thompson et al., 2011).

Given this dependence on low-latency software, understanding the performance implications of different order-book designs is important for both researchers and practitioners. This dissertation evaluates and compares order-book data structures in a realistic benchmarking environment. Detailed technical definitions are provided in Chapter 2; this chapter focuses on the research problem, objectives, and scope of the study.

1.2 Problem Statement and Research Gap

Although there has been significant work on order-book data structures, much of industry practice remains opaque because implementation details are commercially sensitive. Any advantage gained from an optimised order-book design can be worth substantial financial value, so firms rarely disclose architecture decisions in detail. This creates a role for academic benchmarking: providing a reproducible evaluation of the architectures that drive global markets. This study addresses a methodological gap by adopting a two-mode philosophy. While isolated performance is an important metric, it does not capture the "Masking Effect" of the full software path. This dissertation presents a framework that decomposes latency across direct and gateway modes to show where algorithmic wins yield to system-level bottlenecks.

1.3 Aim and Objectives

This dissertation aims to design, implement, and evaluate a C++20 exchange infrastructure for benchmarking order book data structures in high-frequency trading environments. The specific objectives are as follows:

- Define a modular benchmarking framework with interchangeable order-book implementations under a common interface.
- Implement and validate five order-book variants (map, array, vector, pool, and hybrid) with semantic parity across shared test scenarios.
- Evaluate performance in direct mode and gateway mode to separate algorithmic behaviour from end-to-end system overhead.
- Decompose gateway latency into network latency, queue latency, and engine latency to localise bottlenecks.
- Interpret observed differences using data-structure mechanics and hardware-aware reasoning grounded in measured evidence.

To evaluate these objectives, the study uses direct mode and gateway mode, and reports mean latency, P99 latency, throughput, and component-level decomposition of network latency, queue latency, and engine latency against H1–H3.

1.4 Research Hypotheses

The evaluation tests three hypotheses that connect algorithmic structure, allocator strategy, and systems overhead:

- **H1 (Data Structure and Cache Locality):** index-based order-book layouts are expected to outperform pointer-heavy alternatives in isolated operation.
- **H2 (Memory Allocation Limits):** memory pooling is expected to reduce selected allocation costs but not remove the fundamental structural penalties of slower layouts.
- **H3 (System-Level Masking):** in gateway operation, network and queue overheads are expected to compress or hide many internal data-structure performance differences.

For clarity, H1 is treated as supported when index-based designs consistently outperform pointer-heavy designs in direct mode across the core scenarios; H2 is treated

as supported when pooling improves selected allocation-heavy cases without eliminating structural penalties across mixed workloads; and H3 is treated as supported when gateway decomposition shows network and queue costs dominating engine cost, with compressed cross-implementation differences.

These hypotheses and acceptance criteria are defined formally in Chapter 5.

1.5 Scope and Boundaries

This dissertation focuses on software-path benchmarking within a controlled host environment, including direct and gateway execution modes, multiple workload profiles, and comparative analysis across five order-book implementations. The scope includes functional validation, throughput and latency measurement, and component-level decomposition of gateway costs. The scope does not include full physical network-path modelling (for example switch traversal and NIC hardware contention), and absolute latency values should therefore be interpreted as environment-bounded rather than universally transferable.

1.6 Contributions Overview

The dissertation makes four contributions. First, it presents a modular, extensible C++20 exchange framework with interchangeable order-book implementations and a TCP gateway path for end-to-end benchmarking. Second, it provides a reproducible benchmarking suite that separates direct-mode and gateway-mode behaviour while decomposing latency and throughput across pipeline components. Third, it reports empirical evidence on the trade-offs between cache-friendly indexing and pointer-based alternatives, including the bounded benefit profile of memory pooling. Fourth, it demonstrates a systems-level result: in gateway operation, network and queue overheads can dominate total latency and mask differences that are visible in isolated mode.

1.7 Dissertation Structure

Chapter 2 reviews the technical background required for the study, including market microstructure and hardware-aware performance considerations. Chapter 3 presents the architectural design, requirements framework, and benchmarking philosophy. Chapter 4 details implementation choices for the exchange pipeline and order-book variants. Chapter 5 reports functional validation and performance results against the research hypotheses. Finally, Chapter 6 summarises findings, limitations, and future work.

Chapter 2

Background

2.1 High-Frequency Trading and the Limit Order Book

High-Frequency Trading (HFT) is a specialized form of algorithmic trading that executes orders at millisecond or microsecond timescales. HFT strategies are characterised by high order-to-trade ratios, exceedingly short holding periods, and a reliance on co-located servers running directly adjacent to exchange matching engines (Aldridge, 2013). Leading proprietary trading firms, such as Optiver and Citadel Securities, secure direct market data feeds to bypass the latency overhead of standard retail brokers. These firms leverage their ultra-low latency infrastructure to execute a variety of latency-sensitive strategies (Menkveld, 2013). This market-design and speed interaction is also synthesised at the overall market-structure level by O'Hara (2015). This includes spread capture (acting as a market maker by buying at the bid and selling at the ask), avoiding adverse selection (cancelling stale quotes before they are traded against by faster participants), and statistical arbitrage (exploiting fleeting price discrepancies across multiple exchange venues). As Avellaneda and Stoikov (2008) formalise, these strategies depend directly on how quickly the limit order book can be updated and acted upon. This is because even small delays can erode spread capture and increase adverse-selection risk.

A core technological component of this ecosystem is the Limit Order Book (LOB), illustrated in Figure 2.1. The LOB is a centralised electronic record of all active limit orders - instructions to buy or sell a specific quantity of an asset at a specified price or better (Gould et al., 2013). These orders are grouped by price levels and divided into two sides: the *bid* (buyers) and the *ask* or *offer* (sellers). The highest price a buyer is willing to pay is the best bid, while the lowest price a seller will accept is the best ask. The numerical difference between these two values is known as the spread (visually separating the two sides in Figure 2.1), which acts as a primary indicator of market liquidity and volatility. The midpoint between the best bid and best ask provides a theoretical "fair

value" for the asset at any given microsecond. Strategic order submission and cancellation behaviour that drives spread and depth evolution is formalised by Rosu (2009).

	Ask Price	Total Size	
Ask (Sellers)	100.03	50	
	100.02	120	
	100.01	400	← **Best Ask**

Spread = 0.02 | ***Mid-price = 100.00***

	Bid Price	Total Size	
Bid (Buyers)	99.99	250	← **Best Bid**
	99.98	600	
	99.97	150	

Figure 2.1: A stylized Limit Order Book demonstrating price-time priority grouping. The bid (buyers) and ask (sellers) are separated by the spread, establishing the mid-price.

While the official limit order book is maintained by the exchange matching engine, trading firms are responsible for reconstructing their own internal "shadow" books. Exchanges broadcast market events via raw binary protocols like ITCH or FIX, and quant firms must rapidly parse these feeds to rebuild the current book state in their own local memory. The performance of an HFT algorithm depends entirely on how quickly this internal shadow book can process events. This makes the underlying data structures and software architecture paramount.

2.2 Hardware Architecture

To achieve true ultra-low latency, trading systems must be designed to cater directly to the physical realities of the underlying microprocessor. This approach is widely known as Mechanical Sympathy (Fowler, 2011b). The core philosophy dictates that understanding the hardware architecture of a machine enables software to operate in harmony with it, unlocking maximum potential throughput. Rather than treating the CPU as an abstract resource, HFT software treats hardware features - such as cache hierarchies and processor pipelines - as first-class design constraints.

2.2.1 Cache Hierarchy and Memory Access

The cache is the primary hardware component to optimise for when striving for ultra-low latency. It relies on the concepts of temporal and spatial locality to reduce memory operation times. Temporal locality dictates that recently accessed memory locations are highly likely to be accessed again in the near future. Spatial locality assumes that memory locations physically adjacent to a recently accessed address will also be needed soon. This behaviour closely aligns with the Pareto principle, where 80% of memory accesses are typically concentrated within just 20% of an application's data.

To capitalise on this, the memory subsystem is organised into a rigid hierarchical structure, as visualised in Figure 2.2. The L1 cache is the smallest storage layer but resides physically closest to the processor core, offering extremely fast access times of roughly 1 - 2 nanoseconds. Subsequent caches (L2 and L3) are progressively larger but sit further away, incurring proportionately longer fetch times. At the bottom of this hierarchy lies the main system memory (RAM). Fetching data from main memory is significantly slower relative to the cache, routinely taking over 100 nanoseconds and stalling the CPU pipeline entirely (Patterson and Hennessy, 2013).

When the CPU fetches data from memory, it does not retrieve individual bytes; instead, it loads contiguous blocks known as cache lines (typically 64 bytes). This mechanism perfectly leverages spatial locality, ensuring that contiguous data structures - like flat arrays - can be sequentially processed with minimal misses. Conversely, non-contiguous allocations (such as pointer chasing in tree structures) break spatial locality, forcing the CPU to continuously stall while waiting for slow main memory fetches.

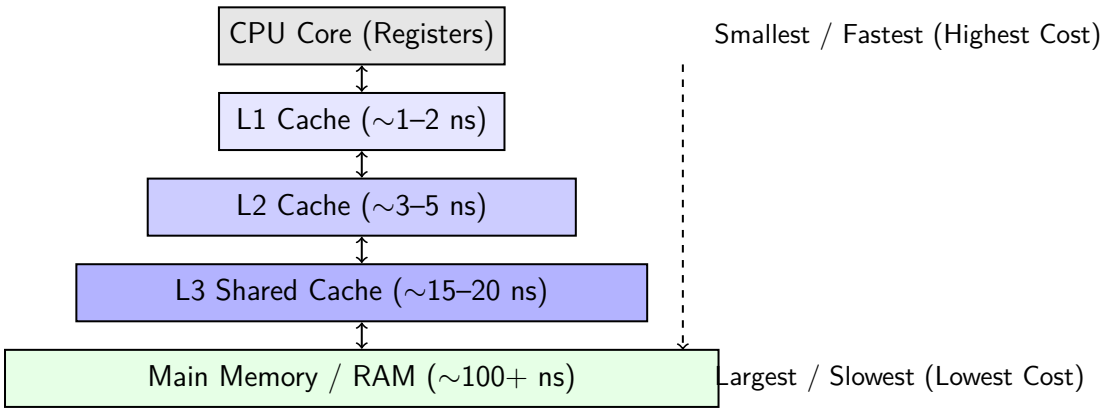

Figure 2.2: CPU memory hierarchy illustrating the explicit size-latency tradeoff. Faster layers are closer to the execution core but exponentially smaller in capacity.

2.2.2 Branch Prediction

Modern CPUs process computation through a specific sequence of stages known as the instruction pipeline, conceptualised in Figure 2.3. While architectures vary, a classic pipeline consists of four fundamental stages: Fetch, Decode, Execute, and Write-back (Patterson and Hennessy, 2013). When the CPU encounters a conditional branch (an `if-else` statement) during the Fetch phase, it does not actually know which path to take until the Execute phase resolves the condition. At this decision point, the CPU has two choices: wait for the execution to finish, or guess the outcome. Waiting would stall the pipeline completely, wasting upward of 15 to 20 clock cycles while the CPU sits idle doing nothing.

To avoid this massive waste of computational time, CPUs rely on a mechanism called a branch predictor. Instead of waiting, the branch predictor guesses which path the program will take and immediately begins speculatively fetching and decoding those instructions. If the guess is correct, execution continues seamlessly with zero wasted cycles. However, if the predictor guesses incorrectly, the CPU must flush all the speculative, half-finished work from the pipeline and restart fetching from the correct address. This pipeline flush incurs a severe penalty, typically costing modern architectures like the AMD Zen 4 between 15 and 20 cycles of completely lost time (Fog, 2025).

Modern predictors, such as the TAGE (TAgged GEometric) or perceptron-based predictors built into current AMD and Intel chips, are incredibly sophisticated and routinely achieve over 95% accuracy (Seznec and Michaud, 2006). Mathematically, the advantage is clear: assuming a 15-cycle misprediction penalty, a 95% accurate predictor yields an average penalty of just 0.75 cycles per branch ($0.95 \times 0 + 0.05 \times 15$), vastly outperforming a strategy of always stalling for 15 cycles.

However, despite this impressive average-case mathematical advantage, unpredictable branches are highly detrimental to ultra-low latency systems. In HFT, average performance is secondary to worst-case tail latency (the outermost response-time percentiles, such as the 99.9th percentile, which dictate the absolute maximum time a system takes to react under stress) (Dean and Barroso, 2013). If complex business logic (such as an unpredictable conditional order fill) resides on the critical path, the inevitable 5% of mispredictions will cause unpredictable, nanosecond-scale execution spikes. Removing unpredictable branches entirely is therefore a primary engineering goal to ensure deterministic, low-variance execution.

2.3 Data Structures for Order Books

The choice of underlying data structure fundamentally dictates the baseline performance of any limit order book. While theoretical algorithmic complexity provides a mathemat-

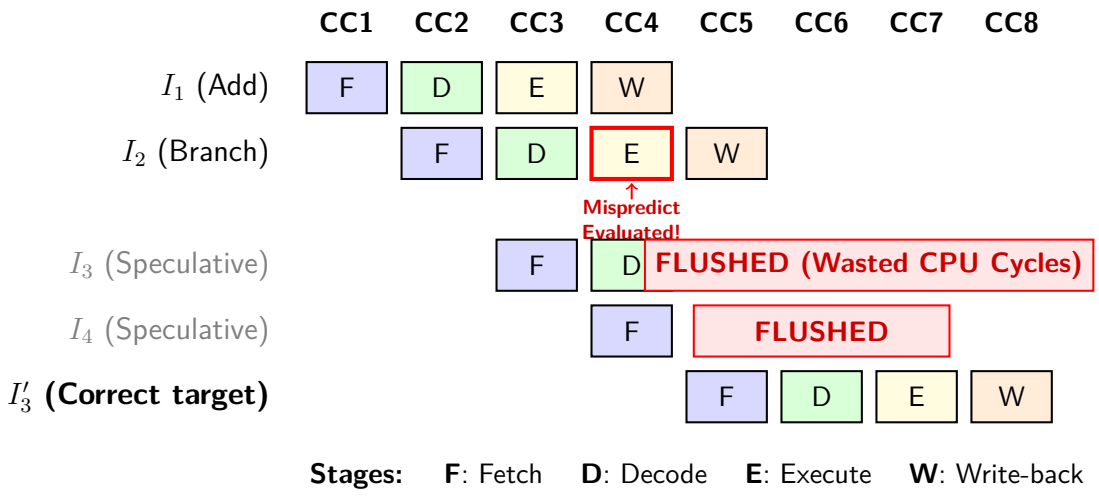

Figure 2.3: A chronological textbook view of a pipeline flush across discrete Clock Cycles (CC). When the branch (I_2) evaluates its condition at the Execute phase (CC4) and discovers a misprediction, all speculatively fetched incoming instructions (I_3, I_4) are forcefully discarded. The pipeline violently stalls until the correct target instructions (I'_3) are fetched and advance through the queue.

ical framework, the physical realities of modern hardware - namely the cache hierarchy and branch predictor - often subvert classical computer science assumptions.

2.3.1 Tree-Based Implementations

Historically, many order books have been implemented using tree-based structures such as Red-Black trees (often conceptually exposed as `std::map` in C++) to manage price levels. These structures offer a theoretical $\mathcal{O}(\log N)$ time complexity for insertions, deletions, and lookups. Their primary advantage is implementation simplicity, as a map intuitively models the domain of a limit order book by directly associating a price level (the key) with a queue of resident orders (the value). Furthermore, trees dynamically and gracefully handle massive, arbitrary price leaps without requiring predefined price boundaries. However, tree nodes are invariably allocated individually on the dynamic heap. As the order book processes thousands of intertwined updates, these nodes become fragmented across disparate locations in main memory. Consequently, traversing a tree to find a specific price level results in rapid, unpredictable pointer chasing. Each pointer dereference frequently triggers a cache miss, forcing the CPU to repeatedly stall while waiting for main memory fetches.

2.3.2 Flat Array Structures

To explicitly combat heap fragmentation, modern HFT architectures favour contiguous, flat structures such as pre-allocated arrays or vectors (Bilokon and Gunduz, 2023). Al-

though inserting an element into a sorted array incurs an $\mathcal{O}(N)$ worst-case complexity (due to shifting elements to maintain chronological order), this metric is theoretically inferior to a tree's $\mathcal{O}(\log N)$. However, because arrays are strictly contiguous in memory, traversing them allows the CPU's hardware prefetcher to eagerly load subsequent elements into the L1 cache before they are even requested (Patterson and Hennessy, 2013). For the relatively small, localised working sets typical of an active limit order book, the sheer speed of contiguous localised memory scans vastly outweighs the algorithmic overhead. In practice, an $\mathcal{O}(N)$ cache-friendly array scan routinely outperforms an $\mathcal{O}(\log N)$ tree traversal for latency-critical workloads. The severe trade-off for this speed is architectural rigidity: to achieve true $\mathcal{O}(1)$ direct array indexing based on price, the book must strictly map discrete *fixed-tick* price intervals to precise internal array offsets, sacrificing the dynamic flexibility of a tree.

2.3.3 Memory Pooling

The final structural consideration is how memory is allocated during live trading. Standard dynamic allocation functions like `std::malloc` or the `new` operator require the operating system to search for free heap space and occasionally request more memory pages from the kernel. This process requires context switching and is inherently non-deterministic. Executing a dynamic allocation on the critical trading path is therefore fatal to predictability because it randomly introduces severe microsecond-scale execution spikes.

As illustrated in Figure 2.4, to bypass the operating system entirely during the trading day, high-performance systems use Memory Pools - large contiguous blocks of memory allocated entirely during system startup. Typically, this is achieved by instructing the OS to persistently map a massive, pre-sized memory region (commonly via the POSIX `mmap` system call) capable of securely holding millions of queued objects without exhausting system bounds (Love, 2013). At runtime, inserting a new order simply claims a pre-allocated object from the local pool via a fast pointer bump, guaranteeing deterministic, $\mathcal{O}(1)$ nanosecond execution.

From a software engineering perspective, this profound architectural change is encapsulated behind standard interfaces. The memory pool acts as a drop-in replacement, exposing the same high-level API methods (e.g., `insert`, `erase`) as a traditional `std::map`, allowing the trading engine to remain agnostic to the underlying memory mechanics. This, however, shifts the burden entirely onto the developer, mandating manual capacity management and the implementation of custom logic to handle aggressive, unforeseen out-of-memory pool exhaustions.

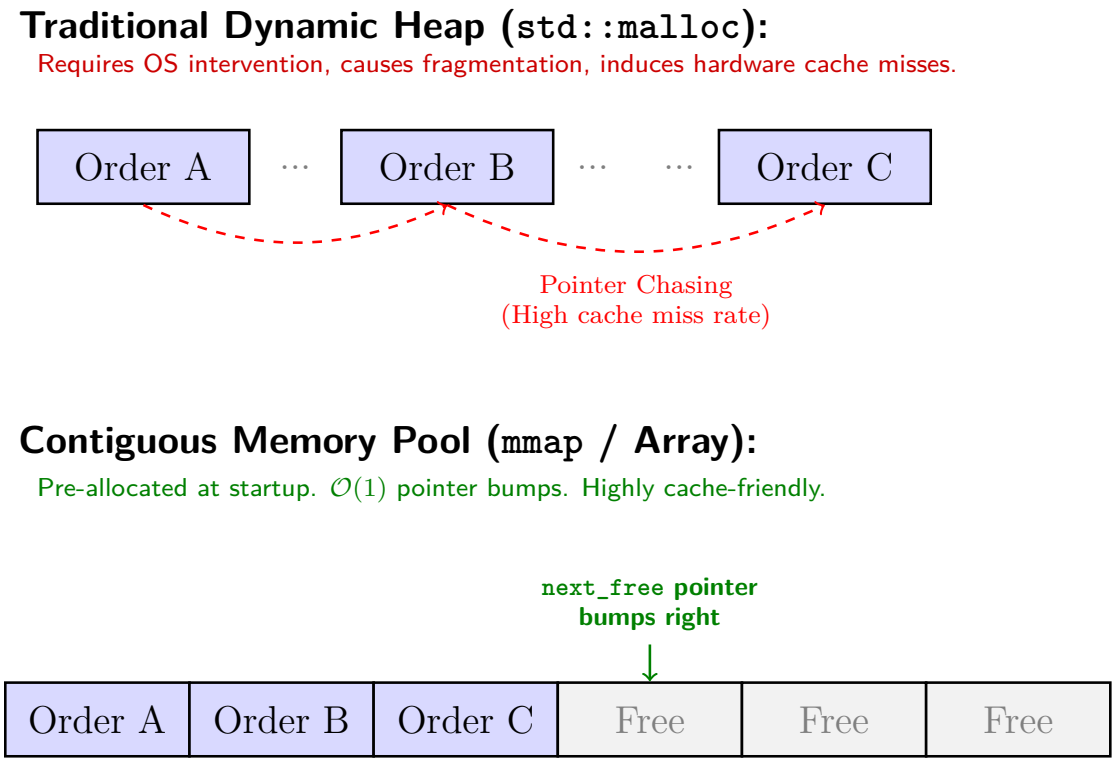

Figure 2.4: Visual comparison of traditional heap allocation versus a Memory Pool. Dynamic allocations mathematically fragment across the heap as orders are rapidly added and deleted, forcing the CPU to fetch disjointed cache lines ("pointer chasing"). A Memory Pool allocates one massive contiguous block upfront, ensuring subsequent active object fetches naturally stream into the L1 hardware cache.

2.4 Lock-Free Concurrent Queues

2.4.1 Exchange Architecture and Queue Modelling

In financial markets, trading firms transmit orders to exchanges across high-speed optical networks. Upon ingress, an exchange must safely and deterministically sequence these concurrent, intertwining arriving messages to enforce absolute chronological fairness. Physical exchanges achieve this sequencing via hardware - performing nanosecond packet timestamping directly on the ingress Network Interface Card (NIC) before funnelling messages through a hardware-level arbiter (Harris, 2002). However, to stress-test the isolated performance algorithms of the matching engine without requiring proprietary networking hardware, our benchmark methodology strictly models this exchange boundary as a system of concurrent software queues. Network-facing threads act as producers that push validated messages into a gateway queue. The matching engine acts as the sole consumer, polling the queue to execute orders sequentially.

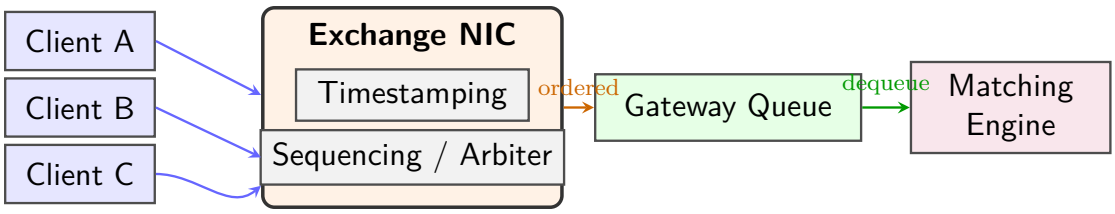

Figure 2.5: Simplified model of exchange message ingress. Packets arrive concurrently from multiple trading clients and are serialised by the Exchange NIC, which applies hardware timestamps and passes messages through an arbiter to enforce strict time-priority ordering. The resulting ordered stream is pushed into a Gateway Queue, from which the single Matching Engine thread consumes sequentially.

2.4.2 SPSC and MPMC Implementations

To evaluate this hand-off boundary, the architecture must forbid the use of operating-system locks. For a 1-to-1 thread mapping (one network thread, one matching engine), a custom Single-Producer Single-Consumer (SPSC) ring buffer is the natural model. As illustrated in Figure 2.6, an SPSC queue coordinates thread access using two entirely independent atomic variables: a *Head* index pushed exclusively by the producer, and a *Tail* index strictly advanced by the consumer.

To validate gateway scalability against concurrent trading clients, the benchmarking architecture must support Many-to-One messaging. For multi-client configurations, the system integrates `moodycamel::ConcurrentQueue` (Desrochers, 2013). This Multi-Producer Multi-Consumer (MPMC) structure allows the evaluation suite to simulate gateway contention from multiple concurrent threads and observe how throughput scales with producer count.

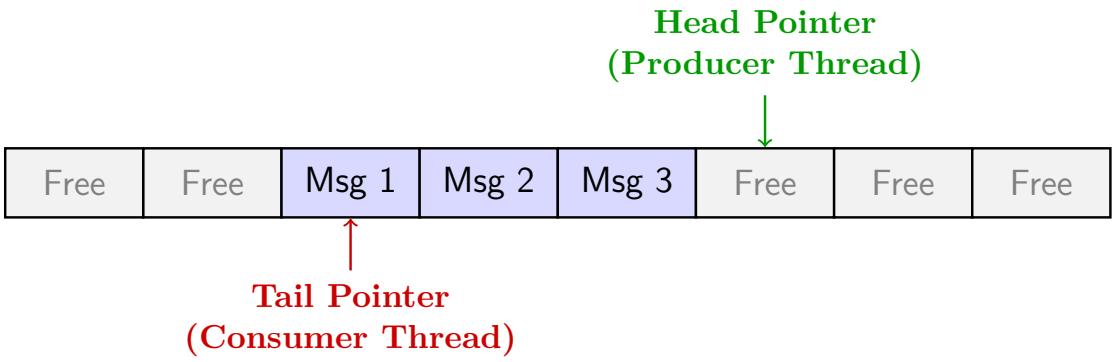

Figure 2.6: A lock-free Single-Producer Single-Consumer (SPSC) ring buffer. The producer actively writes to the Head pointer while the consumer asynchronously trails behind reading from the Tail pointer. Because the two threads concurrently modify distinct atomic variables, they operate entirely lock-free. So long as the underlying logic uses safe modulo arithmetic to wrap edges, the system remains infinitely scalable for isolated threads.

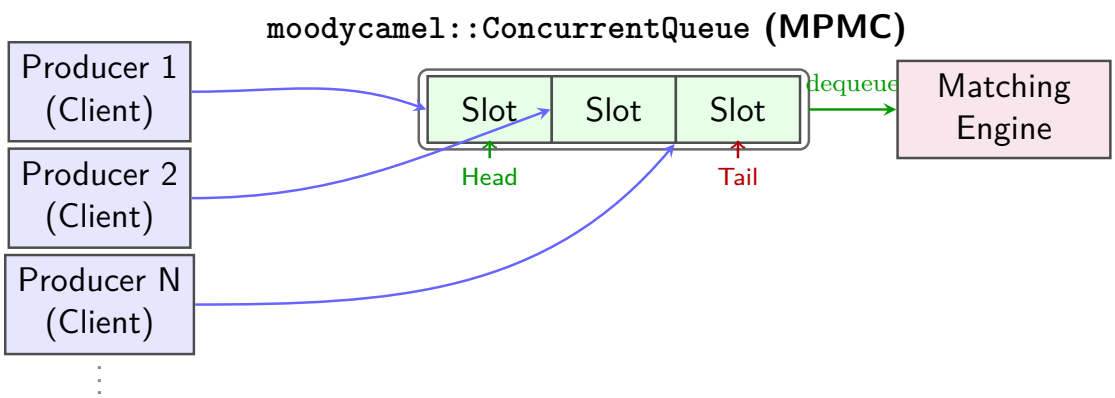

Figure 2.7: Multi-Producer Multi-Consumer (MPMC) queue contention model. Multiple concurrent producer threads (one per trading client) independently push order messages into the shared `moodycamel::ConcurrentQueue`. Atomic Head and Tail indices coordinate access without OS-level locking. A single consumer thread (the Matching Engine) dequeues messages sequentially, allowing the system to measure throughput degradation as producer count increases.

2.4.3 Atomic Operations and Cache Coherence

Lock-free queues remove standard mutual exclusion locks (e.g., `std::mutex`) to avoid the risk of threads being put to sleep by the operating system. By eliminating context switching, they improve determinism and reduce latency spikes during high contention. Instead, they rely on atomic machine instructions, which allow a core to update a variable without relinquishing the CPU.

However, writing to an atomic variable requires a core to obtain exclusive ownership of the memory location. This triggers the hardware's *MESI* protocol (Patterson and Hennessy, 2013), which forces all other cores to invalidate their local cache copies. Communication across the CPU bus to synchronise these states is orders of magnitude slower than a local L1 cache access.

Performance is further degraded by *false sharing*. This occurs when unrelated variables—such as the producer's *Head* index and the consumer's *Tail* index—accidentally occupy the same 64-byte hardware cache line. Because the CPU manages memory in these 64-byte blocks, cores will constantly invalidate each other's cache even though they are updating different indices.

To prevent this, high-performance implementations use padding and C++20 alignment specifiers to ensure each index resides on its own unique cache line, as demonstrated in the following snippet:

```
struct SPSCQueue {
    // Force separation into different 64-byte cache lines
    alignas(64) std::atomic<size_t> head; // Pushed by Producer
    alignas(64) std::atomic<size_t> tail; // Advanced by Consumer
};
```

The spatial management of queue indexes is a prerequisite for scaling lock-free performance on multi-core hardware. These techniques are grounded in foundational non-blocking algorithm theory, notably the single-producer single-consumer and multi-producer queues formulated by Michael and Scott (1996), which define the safety properties required for wait-free ingestion.

The trading industry employs a diverse array of communication protocols, optimised for specific trade-offs between flexibility and performance. These range from flexible, text-based encodings to rigid, dense binary formats. In all cases, these protocols prioritise data integrity, incorporating mandatory checksums and sequence numbers to ensure orders are recovered and reconciled during network instability (FIX Trading Community, 2001). This protocol-level focus is directly relevant in low-latency markets, where high rates of order submission and cancellation increase message volume. In these cases latency from network receive, protocol parsing, and queue waiting becomes a significant part of

end-to-end execution time (Hasbrouck and Saar, 2013).

In our exchange implementation, we use the Financial Information eXchange (FIX) protocol, the global standard for order routing. FIX is a session-based protocol communicated over TCP/IP to ensure reliable, ordered delivery of market messages. It employs a tag-value encoding scheme where each field is represented by a unique numeric tag, an equals sign, and the field's value, as illustrated in Figure 2.8.

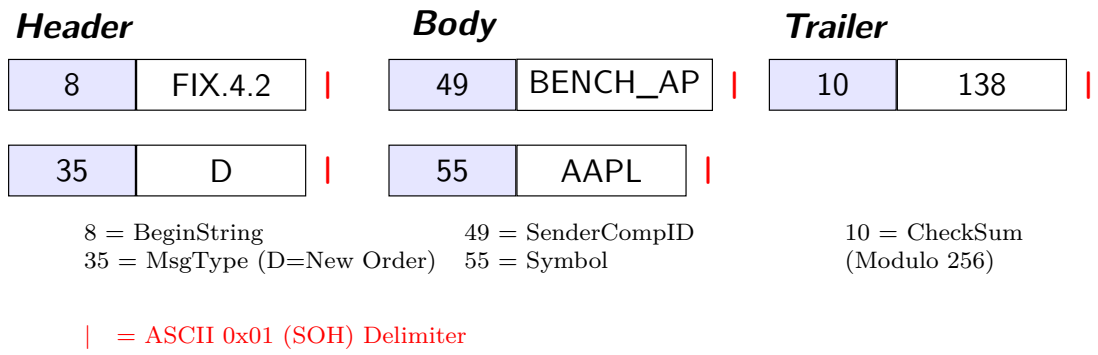

Figure 2.8: Structure of a FIX 4.2 "New Order Single" (MsgType=D) message. The protocol uses a human-readable tag-value encoding where fields are separated by the non-printable SOH character. While flexible, parsing this repeating sequence of ASCII characters induces significantly higher CPU overhead compared to binary protocols like SBE or FAST.

As shown in Figure 2.8, each field is delimited by the `SOH` (Start of Header, ASCII `0x01`) character. The protocol includes a mandatory *Standard Header* containing the message type (Tag 35), a *Message Body* containing order details (e.g., Symbol, Price, Quantity), and a *Standard Trailer* containing a modulo-256 checksum (Tag 10). While the tag-value format is highly extensible and human-readable, parsing these strings necessitates repeated branching and integer conversions, making it significantly slower than modern binary encodings like Simple Binary Encoding (SBE) (Dean and Barroso, 2013).

2.4.4 The Standard Linux Network Stack

When a host receives a FIX message encapsulated within a TCP/IP packet, it must traverse the standard Linux kernel networking stack. This journey, visualised in Figure 2.9, is a rigid, multi-stage pipeline designed for throughput and safety rather than deterministic latency.

Upon arrival at the Network Interface Card (NIC), the packet is written via Direct Memory Access (DMA) to a ring buffer in system memory. The NIC then triggers a hardware interrupt (Hard IRQ), forcing the CPU to pause its current task and branch to the kernel's interrupt handler. The kernel then switches to a Software Interrupt (SoftIRQ) context via the NAPI (New API) polling mechanism to drain the ring buffer, as detailed by Love (2010). The kernel allocates an `sk_buff` data structure to wrap the packet,

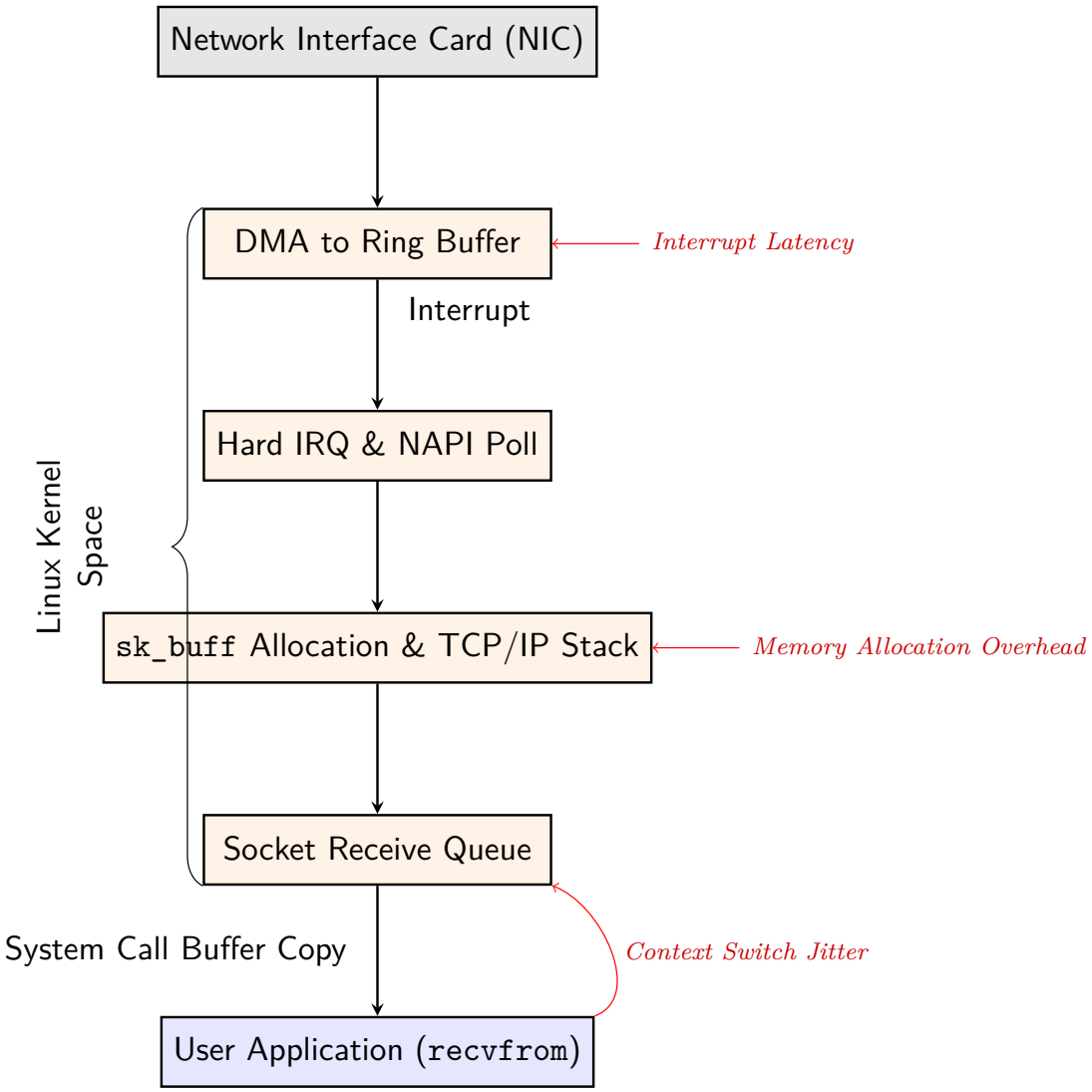

Figure 2.9: The standard Linux kernel network ingress pipeline. A packet must traverse multiple layers of abstraction—including DMA transfers, hardware interrupts, and TCP/IP stack logic—before reaching the application. Each stage, particularly the transition from kernel to user space via system calls, introduces non-deterministic delays that manifest as microsecond-scale jitter in HFT environments.

performs housekeeping and routing (IP layer), and eventually appends the data to the socket's receive queue.

The User Application must issue a system call, such as `recvfrom`, to retrieve the data. This triggers an expensive *context switch* from User Space to Kernel Space, where the kernel copies the data from its internal buffer to the application memory. Because the kernel is responsible for scheduling thousands of unrelated tasks across the CPU, a packet may sit in the socket queue for unpredictable durations if the kernel is busy with book-keeping or other interrupts. This accumulation of interrupts, memory allocations, and context switches is the fundamental source of the microsecond-scale *jitter* (latency variance) that plagues standard network sockets in HFT.

2.4.5 Kernel-Bypass Networking

To mitigate the systemic non-determinism of the Linux kernel, trading firms employ kernel-bypass technologies. Kernel-bypass is an architectural shift that allows the user-space application to talk directly to the NIC hardware, circumventing the OS networking stack, as compared in Figure 2.10.

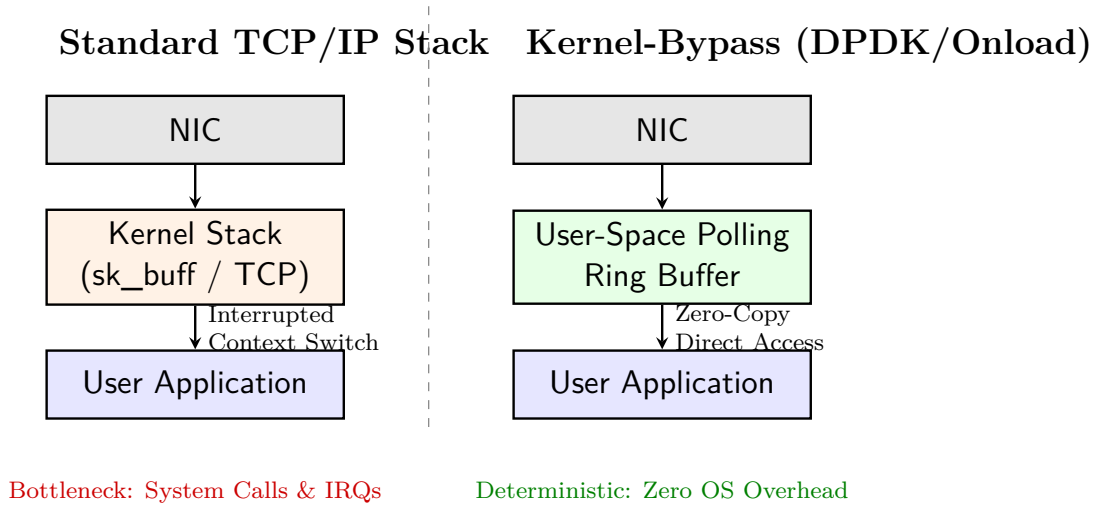

Figure 2.10: A comparison between the standard Linux networking stack and kernel-bypass technologies such as DPDK. While the standard stack requires the kernel to mediate packet delivery—introducing unpredictable context switches—kernel-bypass allows a user-space thread to poll the NIC hardware directly. This eliminates operating system overhead and provides the nanosecond-scale determinism required for HFT execution.

By using specialised libraries such as the Data Plane Development Kit (DPDK) or Solarflare's OpenOnload (Red Hat, Inc., 2012), the application takes ownership of the NIC's hardware resources. Instead of waiting for interrupts, a dedicated CPU core is frequently assigned to "spin" (poll) the NIC's memory-mapped ring buffer in a continuous loop. This achieves *Zero-Copy* delivery, where packets are read directly into application memory without kernel intervention or system calls. This removal of the OS bottleneck

ensures network latency is consistent and deterministic, an essential prerequisite for validating the performance differences of order-book data structures which would otherwise be masked by kernel jitter.

2.5 Related Work

Prior literature can be grouped into market-microstructure theory, systems-design practice, and performance-engineering methodology, each with different strengths and blind spots.

Market-microstructure sources establish why order books matter economically. Harris (2002), Aldridge (2013), and Menkveld (2013) explain liquidity formation, adverse selection, and the strategic role of fast execution. Gould et al. (2013) provides a broad empirical and modelling survey of limit-order-book dynamics. However, these works are not primarily designed as implementation-level benchmarking studies, so they offer limited guidance on how specific in-memory data-structure choices affect nanosecond-scale matching-path latency. Recent exchange-scale evidence by Aquilina et al. (2022) further shows that small latency advantages can have economically significant effects.

Systems-oriented sources describe low-latency architectural patterns but are less controlled from an academic benchmarking perspective. The LMAX work (Fowler, 2011b; Thompson et al., 2011) and more recent systematic evaluations by Bilokon and Gunduz (2023) demonstrate the practical value of single-writer flows, cache-aware patterns, and the "boring" (deterministic) latency profiles required for HFT. Library-level artefacts such as moodycamel (Desrochers, 2013) show strong queue throughput under synthetic conditions, but they do not by themselves establish end-to-end exchange latency behaviour across heterogeneous order-book implementations.

Performance-methodology references provide the measurement principles needed to connect these domains. Gregg (2013) and Dean and Barroso (2013) emphasise observer-effect control, percentile-aware reporting, and bottleneck localisation rather than relying on single aggregate means. This is directly relevant to HFT benchmarking, where tail behaviour and queueing effects can change engineering decisions even when average latency appears similar.

The main gap is therefore not a lack of discussion on HFT speed, but a lack of reproducible, decomposition-driven comparisons that evaluate both algorithmic and end-to-end effects in one framework. Existing literature either focuses on financial behaviour without implementation granularity, or on low-latency engineering patterns without controlled cross-structure comparison in a unified exchange-style pipeline. This dissertation addresses that gap by benchmarking five order-book designs under semantically equivalent workloads in both direct mode and gateway mode, then decomposing network, queue, and engine latency to identify where masking effects occur.

Chapter 3

System Design

This chapter explores the high-level design of the benchmarking suite. Our system is based on a simulated exchange designed to process trades from multiple concurrent clients. To understand the system from first principles, we have engineered a conventional exchange pipeline mostly from scratch. This multithreaded architecture parses FIX messages, passes them to a custom lock-free queue for thread hand-off, and finally dispatches them to the matching engine.

While every component in this pipeline contributes to total system latency, the order book is typically the primary computational bottleneck in a matching engine (Gould et al., 2013). As the stateful core of the exchange, the order book must maintain price-time priority while processing high-frequency updates, making the choice of data structure the most significant factor for engine scalability (Aldridge, 2013). For this reason, this report focuses on exploring the performance trade-offs of varied order book layouts.

When designing a benchmarking suite, it is essential that all measurements are scientific and fair. We maintain this rigour by validating that all components operate with semantic parity, ensuring that latency differences are caused by architectural efficiency rather than logical shortcuts.

3.1 Formal Requirements

To ensure academic rigour and verify that the system performs as intended, we use a formal requirements framework. This approach is inspired by traditional software engineering methodologies for requirements engineering (Sommerville, 2015) and the rigorous systems benchmarking principles advocated by Gregg (2013). These requirements serve as the analytical foundation for the project and provide the objective success criteria for the performance evaluation in Chapter 5. By establishing these benchmarks early, we ensure that the subsequent analysis of order book implementations is grounded in measurable and verifiable standards.

3.1.1 Functional Requirements

ID	Requirement	Testable Criterion
FR-1	Implement five distinct order book data structures	Five implementations present and benchmarkable
FR-2	Support direct-mode benchmarking isolating engine latency	Direct mode runs without network stack involvement
FR-3	Support gateway-mode benchmarking with end-to-end FIX protocol overhead	Gateway mode includes TCP, FIX parsing, and queue handoff
FR-4	Support MPSC contention benchmarking with configurable producer counts	MPSC mode scales across producer configurations
FR-5	Emit structured CSV output for all benchmark runs	CSV files produced with consistent schema per run
FR-6	Include correctness test suite verifying semantic parity across implementations	All implementations produce identical outputs for equivalent inputs

Table 3.1: Functional Requirements (FR)

3.1.2 Non-Functional Requirements

ID	Requirement	Testable Criterion
NFR-1	Correctness: semantic parity across all implementations	No variant has special-case logic biasing results
NFR-2	Fairness: identical workload parameters across all implementations	Same run count, order count, scenario selection for all books
NFR-3	Isolation: direct mode measures only algorithmic/data-structure latency	No transport overhead in direct-mode code path
NFR-4	Realism: gateway mode includes network, queue, parsing, and engine latency	End-to-end timing supports dissertation's masking argument
NFR-5	Reproducibility: pipeline regenerates canonical dataset without undocumented steps	Fresh run produces consistent artifact structure
NFR-6	Statistical rigour: mean, standard deviation, and tail metrics reported	Variance metrics present and consistently derived
NFR-7	Performance isolation: CPU pinning and scheduler isolation applied	Pinning policy documented and verified in results
NFR-8	Portability discipline: absolute latency claims bounded to WSL2/AMD environment	Relative comparisons defensible; absolute numbers environment-scoped
NFR-9	Claim discipline: no measurement overstated beyond evidence	Cache and allocator causality stated as correlation, not proven cause

Table 3.2: Non-Functional Requirements (NFR)

3.2 Design Goals and Constraints

The primary design goal of this benchmarking suite is to minimise measurement overhead and avoid the 'observer effect' (or 'Heisenberg effect'), where the instrumentation itself alters the system's performance characteristics (Gregg, 2013). Ensuring that the act of measuring latency does not significantly inflate the results is essential for maintaining the scientific validity of the data.

Additionally, the project must navigate the technical constraints of the Linux/WSL2

environment. While this virtualised setup introduces additional jitter and overhead compared to bare-metal hardware, these factors are explicitly documented and accounted for throughout the evaluation. By acknowledging these boundaries, we ensure that our conclusions are grounded in the specific realities of the test environment.

3.3 High-Level Architecture

The exchange is designed as a modular system using the 'Ports and Adapters' (Hexagonal) architecture (Velepucha and Flores, 2023; Cockburn, 2005). This approach isolates the core trading logic from external infrastructure, ensuring that the system's performance characteristics remain deterministic and decoupled from transport or logging overheads.

3.3.1 Architectural Pattern: Ports and Adapters

In this design, the core *Matching Engine* (`src/core`) functions as the central "hexagon" containing the domain logic. It remains unaware of whether it is communicating with a real network client or a simulated benchmark feed. Communication with the engine happens through *Ports* - abstract interfaces that define what the engine requires from the outside world.

The most critical port in this system is the `IOrderBook` interface. By defining a stable API for order insertion, cancellation, and matching, the engine can interact with any data structure implementation (the *Adapters*) without modification. This encapsulation is the foundation of our scientific fairness (NFR-2); it allows us to swap an `ArrayOrderBook` for a `MapOrderBook` while keeping the surrounding benchmark harness and engine logic identical.

3.3.2 System Modules

As illustrated in Figure 3.2, the system is composed of four primary modules:

- **Order Generator**: A driving adapter that generates synthetic market workloads for the system.
- **Gateway**: A network-facing adapter that parses FIX messages and manages the transition between network and engine threads.
- **Matching Engine**: The core domain logic that processes the central limit order book.
- **Telemetry**: A driven adapter responsible for capturing nanosecond-resolution metrics at each stage of the pipeline.

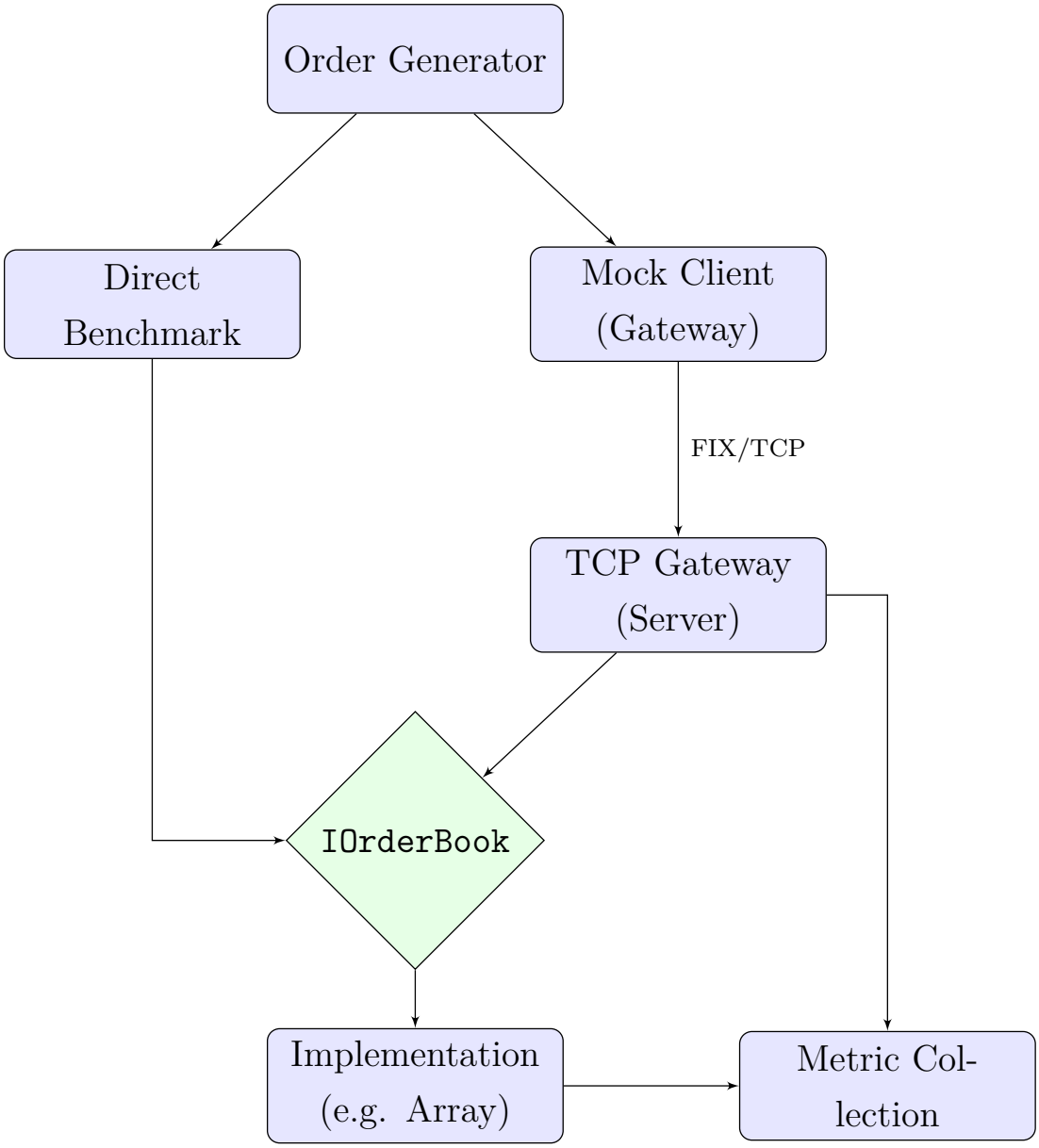

Figure 3.1: Data-Flow Overview: Pipeline stages from generation to telemetry.

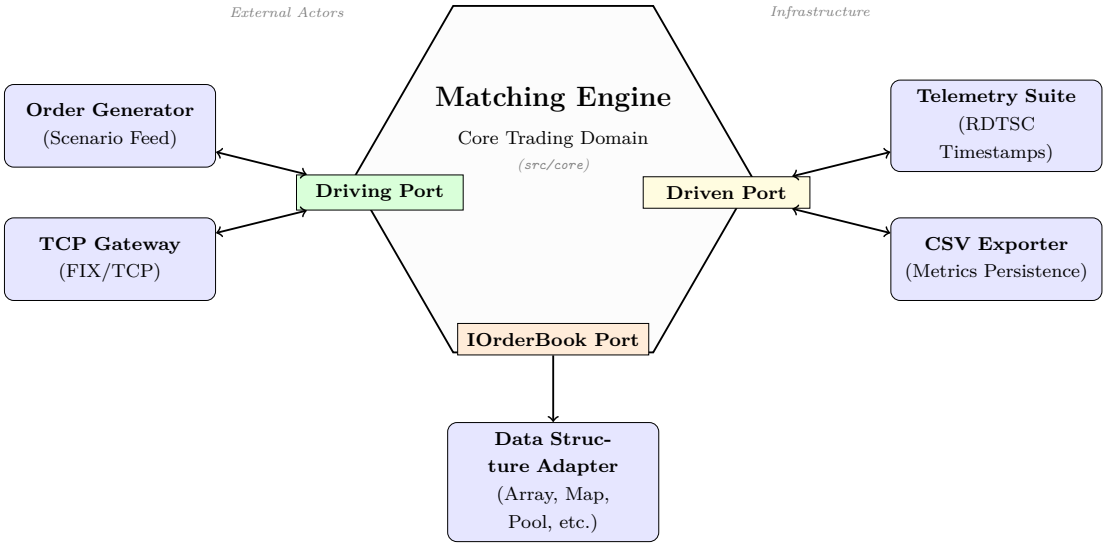

Figure 3.2: Hexagonal Architecture (Ports and Adapters): Clear separation between the matching domain and interchangeable adapters.

3.4 The IOrderBook Interface and Polymorphism

The core of our modular design is the `IOrderBook` interface, implemented as a C++ abstract base class (`src/core/i_order_book.hpp`). This interface defines the contract that any order book data structure must satisfy to be compatible with the matching engine. It standardises essential operations such as order insertion (`addOrder`), cancellation (`cancelOrder`), and the matching logic (`match`).

Using a polymorphic interface ensures that each implementation is treated as a 'black box' by the benchmark harness. This is critical for scientific fairness (NFR-1); it prevents the benchmarking logic from using implementation-specific shortcuts and ensures that every data structure is subjected to the exact same call overhead (i.e. the nanosecond-scale latency of virtual method table lookups). Separating the engine's logic from the book's implementation ensures that our results only reflect the efficiency of the data structures being tested.

3.5 Benchmarking Design: The Two-Mode Philosophy

To validate our hypotheses scientifically, we adopt a dual-mode benchmarking philosophy that separates algorithmic efficiency from overhead. This design is necessary so that our measurements capture both the theoretical potential of our data structures and their relative value in a production-like environment.

The **Direct Mode** serves as a controlled micro-benchmark. It executes the matching engine logic synchronously within the benchmark thread, bypassing all networking and asynchronous layers. This isolation is essential for uncovering the 'pure' performance characteristics of the underlying order book, allowing us to see latency differences that are usually hidden by the surrounding system.

The **Gateway Mode** provides realism by including the full TCP/IP stack (via loopback), FIX protocol parsing, and the asynchronous hand-off between network and engine threads. Measuring the system in this mode is critical for exploring the *Masking Effect* proposed in Hypothesis H3. As we will evaluate in Chapter 5, the constant-time overhead of these networking layers obscures the relative performance advantages of faster data structures. By designing our benchmark suite with both modes, we can differentiate between a structure's peak efficiency and its practical impact on end-to-end latency.

3.6 Concurrency and Threading Strategy

To maintain low latency, the system must separate high-latency I/O operations - such as network packet polling, TCP/IP stack traversal, and FIX message parsing - from the core matching engine. This multi-threaded architecture prevents slower network components from blocking the execution of trades, which is a standard pattern for high-performance sequence processing in industrial matching engines (Thompson et al., 2011).

At the core of the concurrency strategy is the **Producer-Consumer** hand-off. The Gateway thread (Producer) handles the I/O operations described above and pushes parsed orders into a lock-free queue, while the Matching Engine thread (Consumer) pulls and executes them. For the standard gateway mode, we employ a **Single-Producer Single-Consumer (SPSC)** queue, which provides wait-free properties for the engine. In scenarios with multiple concurrent clients, the system can swap this for a Multi-Producer Multi-Consumer (MPMC) adapter from the `moodycamel` library.

To ensure deterministic performance, we use **CPU Pinning** (Thread Affinity). By pinning the engine thread to a specific physical core, we prevent the operating system from migrating the process across different CPUs. This elimination of context-switching jitter and cache-migration overhead is vital for mitigating the *Observer Effect*. It ensures that our measurements represent 'bare-metal' performance as closely as possible within a Linux environment.

Chapter 4

Implementation

4.1 Software Environment and Toolchain

Development of the benchmarking suite required a balance between architectural modularity and strict low-level performance control. We selected C++20 for the engine specifically to use `std::span` and other modern primitives that provide memory safety without the runtime cost of managed pointers. To manage the five discrete order book implementations, we used the `CMake` build system. This allowed us to maintain a modular architecture while ensuring identical compiler flags across all variants.

Key components of the toolchain include:

- **C++20 Primitive Types**: The system uses `std::span` for non-owning views of message buffers. This choice was made to eliminate unnecessary copies during FIX parsing while keeping memory layouts consistent and predictable.
- **POSIX Socket API**: We opted for the raw `sys/socket.h` API over higher-level networking libraries. By bypassing those abstractions, we ensure the gateway's latency is purely a function of the OS kernel stack traversal.
- **Functional Parity Testing**: To ensure that every order book variant produces identical matching results, we built a unit testing suite around the `GoogleTest` framework. This validates that implementations remain functionally consistent, even when their internal memory layouts differ.

4.2 Order Book Implementations

The core of this research is the implementation of five distinct order book strategies, each representing a different point on the efficiency-flexibility spectrum. This section details the technical implementation of the baseline, the static Array, the Hybrid "Hot-Cold" model, and the Memory Pool.

4.2.1 Standard Associative Map (Baseline)

The baseline implementation (`MapOrderBook`) uses `std::map` to link price levels with order queues. We use this to measure the latency cost of node-based allocations and pointer-chasing, as shown in the results in Table 5.6.

4.2.2 Direct-Indexed Array (O(1) Access)

The `ArrayOrderBook` achieves O(1) price retrieval by mapping price levels directly to a contiguous array. While this approach is the fastest possible method for price lookups, it introduces a practical constraint: the book must know its price range at startup. We chose this trade-off to establish a theoretical performance floor for the study, fulfilling the requirements for low-latency execution (FR-2). Mapping prices to indices, as shown in the listing below, is a simple arithmetic operation that remains stable regardless of the book's depth.

```cpp
// src/orderbooks/array_order_book.cpp

Index ArrayOrderBook::priceToIndex(Price price) const {
    // Normalise price to a zero-based index for the array
    return (price - minPrice_) / tickSize_;
}

void ArrayOrderBook::addOrder(const Order &order) {
    if (!isValidPrice(order.price)) return;

    // Direct access to the price level's memory location
    Index index = priceToIndex(order.price);

    if (order.side == Side::Buy) {
        bidLevels_[index].push_back(order);
        activeBidLevels_[index] = true;
        if (order.price > cachedBestBid_) cachedBestBid_ = order.price; // O(1)
    } else {
        askLevels_[index].push_back(order);
        activeAskLevels_[index] = true;
        if (order.price < cachedBestAsk_) cachedBestAsk_ = order.price; // O(1)
    }
}
```

Listing 4.1: O(1) Price-to-Index Mapping

This design achieves FR-2 by providing deterministic latency, independent of book depth. However, it requires prior knowledge of the price range, a constraint common in fixed-range instruments.

4.2.3 Hybrid "Hot-Cold" Vector-Map

To balance the speed of contiguous memory with the flexibility of associative storage, the `HybridOrderBook` implements a tiered memory model. It maintains the "Hot" price levels (those closest to the spread) in a sorted `std::vector` for cache-friendly matching, while "Cold" levels (out-of-the-money) are stored in a `std::map`.

```
// src/orderbooks/hybrid_order_book.cpp

void HybridOrderBook::addOrder(const Order &order) {
    // Check if the price level is within the performance "Hot Path"
    if (isCloseToSpread(order.price, order.side == Side::Buy)) {
        if (hotBids_.size() >= maxHotLevels_) {
            demoteFromHot(true); // Evict furthest level to Map
        }
        addToHot(order, true); // Insert into contiguous Vector
    } else {
        addToCold(order, true); // Store in associative Map
    }
}
```

Listing 4.2: Hot/Cold Proximity Check

This model balances the flexibility of a traditional order book (FR-1) with higher performance for price levels immediately surrounding the spread. The impact of the hot-path size on overall matching throughput is analysed in Section 5.4.1.

4.2.4 Pool-Allocated Order Book (Slab Allocation)

Runtime heap allocations are a primary source of latency jitter in dynamic systems. The `PoolOrderBook` mitigates this by using a pre-allocated slab of `OrderNode` structures. By managing an internal free list, the engine can add or cancel orders without requesting memory from the OS kernel, ensuring the matching path remains deterministic.

```cpp
// src/orderbooks/pool_order_book.cpp

Index PoolOrderBook::allocateSlot() {
    // O(1) Pop from the pre-allocated free list
    Index index = freeHead_;
    freeHead_ = orders_[index].nextFree;

    // Initialise node state without new/malloc
    orders_[index].next = NULL_IDX;
    orders_[index].prev = NULL_IDX;
    return index;
}

void PoolOrderBook::freeSlot(Index idx) {
    // Return node to pool for immediate recycling
    orders_[idx].nextFree = freeHead_;
    freeHead_ = idx;
}
```

Listing 4.3: Slab-Allocation and Free List Management

By reusing memory slots, we keep the matching logic deterministic. This mitigates allocation jitter and helps verify the reduced tail latency analysed in Section 5.4.1.

4.3 The TCP Order Gateway

To evaluate the system under realistic network conditions for H3, we implemented a high-performance TCP gateway. This component connects external FIX clients to the internal core through a lock-free interface.

4.3.1 Protocol Specification and Parsing

Our FIX implementation focus was on parsing speed rather than full protocol coverage. The gateway uses a non-allocating incremental parser that processes raw buffers as they arrive. As the listing below demonstrates, we used a sliding window technique to handle fragmented TCP packets, satisfying the realism requirement of NFR-4 while keeping the matching core separate from network ingestion.

```cpp
// src/network/tcp_order_gateway.cpp

void TCPOrderGateway::clientHandler(int clientSocket) {
```

```
    std::vector<char> buffer(4096);
    size_t offset = 0; // Tracks partial message data

    while (running_) {
        // Read data into the buffer starting after the leftover data
        ssize_t bytes = recv(clientSocket, buffer.data() + offset,
                             buffer.size() - offset, 0);

        size_t totalBytes = offset + bytes;
        size_t processed = 0;

        while (processed < totalBytes) {
            size_t consumed = 0;
            std::span<const char> data(buffer.data() + processed,
                                       totalBytes - processed);

            auto order = FIXParser::parse(data, consumed);
            if (consumed == 0) break; // Incomplete message, wait for more

            if (order) {
                order->receiveTimestamp = getCurrentTimeNs();
                while (!orderQueue_.push(*order)) std::this_thread::yield();
            }
            processed += consumed;
        }

        // Shift remaining data to the front for the next recv()
        if (processed < totalBytes) {
            std::memmove(buffer.data(), buffer.data() + processed,
                         totalBytes - processed);
            offset = totalBytes - processed;
        } else {
            offset = 0;
        }
    }
}
```

Listing 4.4: Incremental FIX Parsing Loop

4.3.2 The Loopback Simulation Model

To measure kernel networking overhead without the jitter of hardware NICs, we run the gateway on the loopback interface (`127.0.0.1`). This allows us to isolate the "Wire-to-Engine" latency analysed in Section 5.5.

4.4 High-Performance Concurrency

The system employs a multi-threaded architecture to separate high-latency I/O from the latency-sensitive matching core.

4.4.1 Wait-Free SPSC Queue

For Direct and single-client Gateway modes, we use a custom Single-Producer Single-Consumer (SPSC) queue. This implementation relies on a contiguous ring-buffer and atomic head/tail pointers with `std::memory_order_acquire/release` semantics (Michael and Scott, 1996), ensuring that the matching engine does not block in this design due to lock contention.

In multi-producer scenarios (FR-4), the system swaps the SPSC queue for the `ConcurrentQueue` library (Desrochers, 2013). We opted for this established lock-free implementation to ensure our results reflect the underlying data structure performance rather than our own queue implementation logic. We evaluate this MPMC transport under contention in Section 5.6.

4.5 Low-Level Performance Utilities

To ensure nanosecond-level accuracy and deterministic execution across benchmarks, the system includes a set of low-level utilities for precise timing and thread management.

4.5.1 Nanosecond Precision Timing

To achieve the nanosecond granularity required for order book benchmarking, the system employs the `std::chrono` high-resolution clock. While cycle-level precision is typically associated with the CPU's Time Stamp Counter (`RDTSC`), our implementation uses the standard C++ library to ensure portability across environments (specifically WSL2) while maintaining the precision necessary for the evaluation in Chapter 5.

```
// src/utils/rdtsc.hpp

inline uint64_t getCurrentTimeNs() {
```

```
    // Cross-platform fallback. In production, this is replaced
    // by __builtin_ia32_rdtsc() for cycle-level accuracy.
    return std::chrono::duration_cast<std::chrono::nanoseconds>(
                std::chrono::high_resolution_clock::now().time_since_epoch()
            ).count();
}
```

Listing 4.5: Nanosecond Precision Timing

4.5.2 Thread-to-Core Affinity (CPU Pinning)

To eliminate the performance jitter caused by the Linux scheduler migrating threads across physical cores (the "Context Switching Jitter"), we implemented CPU pinning as shown in Listing 4.5.

```
// src/utils/thread_pinning.hpp

inline bool pinToCore(int core_id) {
#if defined(__linux__)
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);       // Clear the CPU mask
    CPU_SET(core_id, &cpuset); // Set the bit for the target core

    // Request the OS scheduler to bind this thread to the selected core
    pthread_t thread = pthread_self();
    return pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset) == 0;
#endif
    return false; // Fallback for unsupported OS
}
```

Listing 4.6: POSIX Thread Pinning Strategy

By binding the matching engine thread to a specific physical core, we eliminate "Context Switching Jitter" where the OS scheduler migrates a thread between CPUs. This ensures that the engine's working set remains "hot" in the local L1/L2 caches, maintaining the deterministic execution time required to validate our performance claims. This pinning helps satisfy NFR-4 by making sure our measurements are not skewed by operating system interference.

4.6 Automation and Data Collection

The suite uses a Python-based driver to sweep across the benchmark scenarios defined in Chapter 3. This driver coordinates the execution of each order book variant and manages the serialised CSV output required for the detailed evaluation in Chapter 5.

Chapter 5

Testing and Evaluation

5.1 Experimental Methodology

This chapter evaluates five order book implementations against three core research hypotheses (H1, H2, and H3). The benchmarking environment is isolated via thread-pinning to the canonical x86_64/WSL2 host (Ryzen 7600X) to ensure reproducible nanosecond-level precision. Given that WSL2 is a virtualised, Hyper-V-managed utility VM (Microsoft, 2024), there is an inherent "virtualisation tax" and network overhead. However, this additional latency is consistent across all tests and does not impact the validity of our results.

To isolate systemic and algorithmic latency, the benchmarking suite incorporates two distinct modes. **Direct mode** focuses on isolating algorithmic performance by using thread-pinning and removing transport overhead, thereby adhering to the isolation standard defined in NFR-3. This configuration allows for the capture of performance trends within the order book implementations themselves with minimal external interference. Conversely, **Gateway mode** provides a realistic evaluation by incorporating network transport, queueing, FIX parsing, and matching engine processing. This offers a shadow-to-engine view of the end-to-end pipeline in accordance with NFR-4.

When collecting data, we capture significant statistics for four core operations: **insert, cancel, lookup, and matching**. For each of these, we calculate a set of metrics, including **mean latency, standard deviation, P99 tail latency, and throughput**, adhering to NFR-6. In gateway and MPSC modes, we also decompose these latencies into **network, queue, and engine processing** components to isolate performance bottlenecks. An important attribute of our data is to make it reproducible; this is achieved by a well-documented benchmarking suite that generates a canonical dataset every time it is run, adhering to NFR-5. To guarantee scientific rigour, we rerun our benchmarks multiple times per configuration, where a configuration is defined by `Mode/Scenario/Book` (and `Producers` for MPSC rows). We then use the run set for that configuration as the

basis for statistical summary. In this dataset, direct-mode configurations use $n = 30$ samples, gateway configurations use $n = 20$ samples, and MPSC configurations use $n = 30$ samples for each `Scenario/Book/Producers` setting (with producers $\in \{1, 2, 4, 8\}$); each sample run still processes the full per-run order volume. We use a larger n for direct mode because those runs are cheaper to execute, while gateway runs include full network/FIX/queue stages and therefore take longer per replicate; this balances precision against total campaign time. These replicate counts provide stable summary statistics for this chapter's hypothesis-level analysis and narrow uncertainty intervals around the key cross-implementation comparisons.

To quantify estimation uncertainty, we report 95% confidence intervals using:

$$\bar{x} \pm t_{0.975,\, n-1} \frac{s}{\sqrt{n}},$$

where $\bar{x}$ is the sample mean, s is sample standard deviation, and n is sample size. The term $t_{0.975,\, n-1}$ is the 97.5th percentile of the Student-t distribution with $n - 1$ degrees of freedom (the two-sided 95% critical value). For hypothesis interpretation, this gives a practical rule: when two implementations are compared within the same scenario, non-overlapping intervals indicate a stronger signal that the observed latency difference is not just run-to-run noise.

Finally, to evaluate these findings, we refer to the original research hypotheses (H1, H2, and H3) as our primary analytical framework. To maintain structural rigor, each performance claim in the subsequent sections is mapped to its corresponding hypothesis and any relevant Non-Functional (NFR) or Functional (FR) requirement IDs.

Refined Research Hypotheses

H1 (Data Structure and Cache Locality). Order books that use index-based layouts (like arrays) will be significantly faster than search trees or sorted vectors. This advantage is primarily driven by improved CPU cache behaviour during high-frequency updates.

Acceptance criteria: The array implementation shows the lowest mean latency in direct mode across most scenarios, and performance counters show lower memory-access pressure compared to others.

H2 (Memory Allocation Limits). While memory pooling can reduce specific allocation costs, it will not overcome the fundamental performance bottlenecks of a slow data structure.

Acceptance criteria: The pool implementation shows statistically significant gains ($p < 0.0125$ with Bonferroni correction) over the standard map, but does not reach the performance levels of the array where sequential searching is the main bottleneck.

H3 (System-Level Masking). In a networked (gateway) environment, the overhead of the network and system queues will account for the vast majority of end-to-end latency, masking the algorithmic efficiency of the order book implementation.

Acceptance criteria: For the primary non-vector implementations (array, map, pool), the combined transport overhead (network and queue) accounts for the significant majority ($> 85\%$) of total end-to-end latency.

5.2 Functional Validation

5.2.1 Core Logic and Semantic Parity

To verify that our order books follow semantic parity (NFR-1), we used parametrised contract testing (Fowler, 2011a) (FR-6). Rather than testing our order books individually, we applied the same tests to each implementation in the form of a contract. We achieved this by using Google Test's parametrised suites (Google, 2024), ensuring each order book was verified against identical market scenarios (including happy paths, partial fills, and edge cases). This is critical as it guarantees that any subsequent performance differences observed are due to architectural efficiency rather than logical differences, ensuring fairness in our comparisons (NFR-2).

This approach follows the **Ports and Adapters architecture** (Cockburn, 2005), using contracts to reinforce the **Liskov Substitution Principle** (Liskov, 1987). Beyond validating correctness, this strengthens code maintainability. Utilizing the LLVM coverage toolchain (LLVM Project, 2024), the project achieved 100% coverage across branches, lines, regions, and functions in the core order book logic (summarized in Table 5.1), proving that the benchmarking results are derived from fully validated and semantically equivalent components.

Metric	Coverage Percentage
Function Coverage	100.00% (96/96)
Line Coverage	100.00% (1355/1355)
Region Coverage	100.00% (572/572)
Branch Coverage	100.00% (370/370)

Table 5.1: Code Coverage Results for Core Order Book Logic

To evaluate the system's performance limits across different conditions, we integrated a range of market scenarios. These act as a rigorous functional test, revealing how each order book performs under varying loads. These scenarios include:

- **Full Aggressive Matches**: To verify the matching logic's ability to correctly exhaust liquidity and remove orders across multiple price levels.

- **Nested Limit Orders**: To test the data structure's efficiency in maintaining price-time priority across complex book depth.
- **Invalid Entry Handling**: To verify that zero-quantity or illegal price orders are safely rejected without corrupting the internal state.

We compared all five implementations against a **golden reference** derived from the `MapOrderBook` implementation. As the most straightforward translation of limit order book logic, it serves as the benchmark to ensure that the more complex, high-speed implementations maintain identical logical behaviour.

To ensure the statistical validity of the most complex test case, the **mixed** scenario, we define three price-proximity tiers relative to the market midpoint. These consist of **near-spread** activity (orders within ±6 ticks of the mid-price), **deep share** (±10 to ±50 ticks), and **extreme share** (outliers at ±400 to ±600 ticks), where a 'tick' represents the minimum price increment of the exchange. As shown in Figures 5.1 and 5.2, this configuration is validated by capturing the branch probabilities within the order generator to confirm the workload maintains a heterogeneous distribution rather than collapsing into a single regime (NFR-1). This level of transparency in the benchmark generation ensures that all subsequent performance claims are strictly bounded by known evidence, adhering to NFR-9.

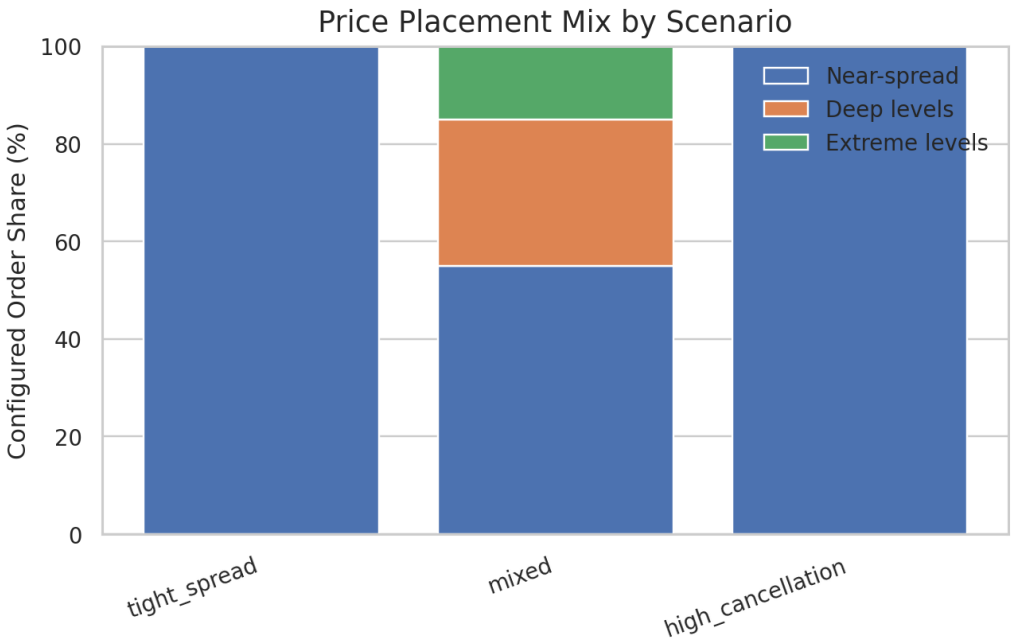

Figure 5.1: Mixed-scenario price distribution: the 55/30/15 split across near-spread, deep, and extreme price levels confirms the workload generator produces a heterogeneous order flow rather than collapsing into a single regime (NFR-1, NFR-9).

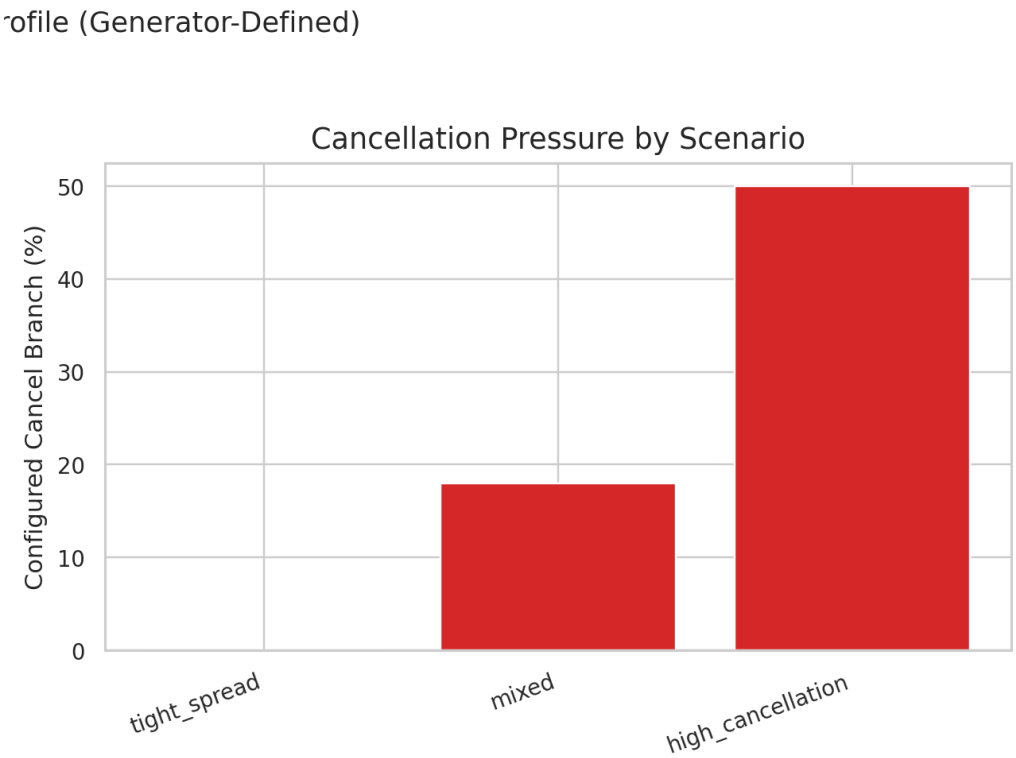

Figure 5.2: Mixed-scenario cancellation pressure: the 18% cancellation rate is verified against its theoretical configuration, validating benchmark integrity (NFR-1, NFR-9).

5.2.2 System Pipeline and Integration

Stepping back from the order book implementations, it was essential to validate the entire system pipeline to ensure that functional correctness is maintained from the point of entry to the point of execution. This pipeline, illustrated in Figure 5.3, spans from the initial order request submitted via **FIX 4.2** (FIX Trading Community, 2001) by a gateway client to the eventual consumption by the matching engine.

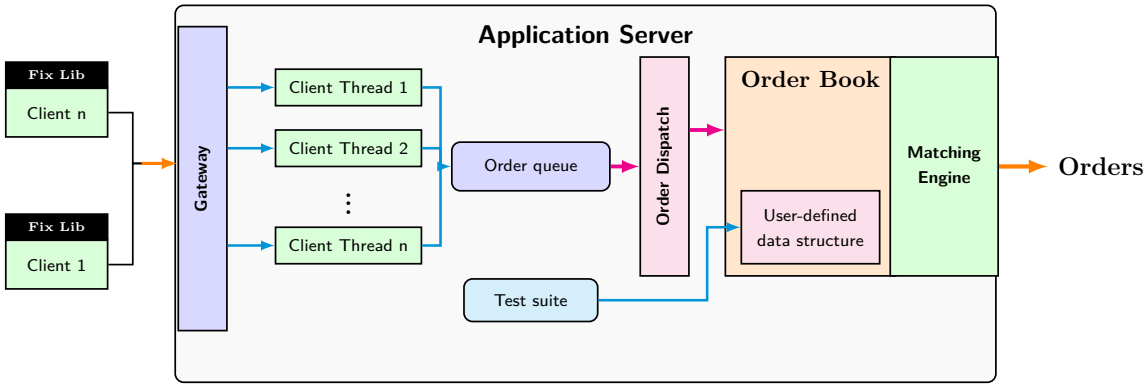

Figure 5.3: Full System Architecture: From Gateway Client to Matching Engine

The validation process confirms that incoming **New Order Single (Tag 35=D)**

messages are correctly parsed and dispatched into a cache-aligned, **lock-free SPSC InputQueue**. To maintain strict **traceability**, we capture the client's unique order ID (Tag 11) and original transmission time (Tag 60 (Onix Solutions, 2019)) and pair them with an internal **gateway receipt timestamp**. This "quad-timestamp" strategy ensures the integrity of the end-to-end flow; by verifying these IDs across the multi-threaded network layer and the single-threaded dispatching loop, we confirm that no orders are lost or reordered during transport. The validation process confirms that the **New Order Single (Tag 35=D)** message is correctly mapped to internal C++ structures as summarized in Table 5.2.

Tag	FIX Field Name	Example Value	Internal C++ Mapping
35	MsgType	D	`NewOrderSingle`
11	ClOrdID	1001	`OrderId id`
54	Side	1 (Buy)	`Side::Buy`
38	OrderQty	7	`Quantity quantity`
44	Price	130	`Price price`
60	TransactTime	123456789	`Timestamp sendTimestamp`

Table 5.2: Mapping of FIX 4.2 Application Tags to Internal C++ Order Structures

5.2.3 Scaling and Stress Testing

While the primary architecture uses a custom-built, wait-free **Single-Producer Single-Consumer (SPSC)** queue for learning and simplicity, we also needed to test how the system behaves under producer contention. To achieve this, we integrated an open-source **Multi-Producer Multi-Consumer (MPMC)** queue from the **Moodycamel** library (Desrochers, 2013). By running this queue in a single-consumer configuration, we were able to evaluate how the system behaves under producer contention from multiple concurrent clients pushing into the matching engine. Both queue implementations were subjected to the same functional validation to ensure that the choice of transport did not affect the final order book state.

The `FIXParser` was also verified to ensure it behaves correctly during fragmentation (where the network stream is split into multiple packets) by successfully reassembling messages. The queue was tested for back-pressure handling; if the 1024-deep queue reaches capacity, a `push()` call will fail, with the expected behaviour being that the client is alerted rather than the system crashing. Finally, Moodycamel's MPMC queue was integrated to ensure that the system maintains stability and does not significantly degrade in performance when multiple concurrent clients are querying the engine at the same time.

5.2.4 Gateway Simulation Boundaries and Validation

To ensure the end-to-end performance results are technically sound, we will define the boundaries of the Gateway simulation. As identified in Chapter 4, the benchmark utilises a **Loopback (AF_INET)** configuration where the `MockClient` and `TCPOrderGateway` communicate via the local interface (`127.0.0.1`).

This setup was validated to confirm that it correctly exercises the full **Linux TCP/IP stack**, including the overhead of system calls (`send`, `recv`, `select/poll`), kernel buffer copies (`sk_buff`), and the associated context switching jitter. However, because the client and server reside on the same physical host, this environment eliminates the latency of physical network media (cables) and NIC hardware processing (e.g., PCIe bus latency). Consequently, the absolute results recorded in the "Net" component represent a **theoretical lower-bound** for system latency, effectively isolating the "software networking tax" from the "physical hardware tax."

5.3 Throughput Results

This section evaluates hypotheses H1 and H3 by analysing system throughput in direct and gateway modes.

The throughput results in direct mode work to verify H1, which claims that order books with index-based layouts will be faster than those with search trees or sorted vectors. We can see evidence for this claim by observing the greatest direct throughput of 1,442,735 ops/s achieved by the **ArrayOrderBook**. It remains clearly ahead of the next tier (**PoolOrderBook** at 810,382 ops/s and **MapOrderBook** at 799,534 ops/s), while **VectorOrderBook** is lowest at 693,564 ops/s. These direct-mode values are averaged across seven scenarios (tight spread, fixed levels, dense full, sparse extreme, worst case FIFO, mixed, and high cancellation), each of which exercises a mix of insert, cancel, lookup, and match operations, adhering to NFR-6. Although **ArrayOrderBook** wins here, it is not without its downsides. When using index-based order books, the implementer is forced to define a fixed range of valid prices and a discrete tick size. Additionally, logic must be provided to handle orders whose prices fall outside the maximum and minimum values of the order book, or that do not fall exactly on a price level. These restrictions may limit the flexibility and accessibility of the exchange compared to standard pointer-based designs. The second-place **PoolOrderBook**, by contrast, imposes no such price-range constraints. Its competitive throughput can be attributed to its custom memory pool, which eliminates dynamic heap allocation on the critical path and reduces allocator overhead compared to the pointer-chasing traversal of **MapOrderBook**. For these best-versus-worst direct comparisons, the 95% intervals are clearly separated in the generated summaries, so the performance gap is unlikely to be explained by run-to-run

noise alone.

To rigorously evaluate whether the measured latency differences are the result of structural design choices or merely random measurement noise, we perform formal pairwise comparisons using Welch's t-test (Welch, 1947). This analysis tests the null hypothesis (H0): the assumption that there is no real performance difference between two implementations, and any observed gap is due to chance. To reject this hypothesis, we calculate a t-statistic, which interprets the "signal" (the difference in mean latency) relative to the "noise" (the variance or jitter in the data). A larger t-statistic indicates that the performance gap is many times larger than the underlying randomness of the system.

Welch's variant of the t-test is specifically selected because it does not assume equal variances between the compared groups. In low-latency software, different algorithms (e.g., array indexing vs. pointer-chasing in a pool) often exhibit different levels of "tail jitter," making this robust approach necessary for scientific accuracy (Welch, 1947).

Furthermore, because we are making four separate comparisons (comparing the Array implementation against the Map, Pool, Vector, and Hybrid books), we must account for the "multiple comparisons problem." If we use a standard 5% significance level ($\alpha = 0.05$) for each test, the cumulative probability of finding a "false positive" by pure luck increases across the four tests. To maintain a strict overall confidence level, we apply the Bonferroni correction (Dunn, 1961). This involves dividing our target significance ($\alpha = 0.05$) by the number of comparisons ($m = 4$), resulting in a more conservative adjusted threshold of $\alpha_{adj} = 0.0125$.

Across all primary scenarios, the results provide overwhelming evidence against the null hypothesis. In the mixed regime, comparing the ArrayOrderBook to the MapOrderBook yields a t-statistic of t=-75.09 (df = 43.87, p<0.0001). This value indicates that the "signal" of the array implementation's superiority is 75 times stronger than the measurement "noise." A similarly extreme separation is observed in the `dense_full` scenario against the map alternative (t=-119.14, df = 30.55, p<0.0001). Even in the highest-performing non-array tier (`tight_spread`), the array implementation maintains a clear statistical lead (t=-44.92, p < 0.0001). This formal analysis confirms that the memory-layout optimisation in the array implementation delivers a genuine, statistically significant latency reduction that cannot be attributed to environment jitter.

The gateway mode results support H3. In a realistic exchange with overheads in the form of the network and queues, these additional latencies mask the performance differences between order books. In direct mode, the throughput spread across all implementations reached 51.9% (1,442,735 vs 693,564 ops/s). In gateway mode (for the three available scenarios), the spread is 33.3% (302,223 vs 201,457 ops/s), with non-vector implementations still clustered more tightly than direct mode.

We restricted gateway benchmarks to three representative scenarios (tight_spread, mixed, and dense_full) to focus our available compute-time on high-fidelity replicates

Implementation	Direct Mode (ops/s)	Gateway Mode (ops/s)
ArrayOrderBook	1,442,735	302,223
MapOrderBook	799,534	299,044
VectorOrderBook	693,564	201,457
HybridOrderBook	717,375	283,384
PoolOrderBook	810,382	297,033

Table 5.3: Mean Throughput Across Implementations in Direct Mode (7 scenarios) and Gateway Mode (3 scenarios: `tight_spread`, `mixed`, `dense_full`)

for the most significant market regimes. Given that each gateway run requires full-stack network and queue processing, this focused selection balances campaign duration with the statistical precision required for H3. The data indicates that this convergence is a direct consequence of the network component consuming approximately 88–96% of total end-to-end latency for the primary non-vector books (array/map/pool; see Table 5.8), in accordance with NFR-4. The data shows that the ordering structure of the engine becomes largely irrelevant when the transport stack dominates the pipeline.

Implementation	Net Share (%)	Queue Share (%)	Engine Share (%)
ArrayOrderBook	95.88	4.12	0.0030
MapOrderBook	91.55	8.43	0.0240
PoolOrderBook	88.39	11.59	0.0245

Table 5.4: Compact gateway decomposition evidence for primary non-vector books (derived from Table 5.8)

Table 5.4 clarifies the masking effect: for the primary non-vector books (array, map, pool), the network alone consumes 88–96% of total end-to-end latency, while engine compute remains near zero as a percentage share.

To make interval evidence explicit for key direct comparisons, Table 5.5 reports 95% confidence intervals for representative shallow, mixed, and dense direct-mode scenarios. Across these cases, the array interval remains clearly separated from non-array implementations, which supports the corresponding direct-mode hypothesis interpretation.

5.4 Latency Analysis - Direct Mode

This section provides a deeper analysis of the per-operation latencies across all order book implementations, strictly in direct mode to isolate data structure behaviour from transport overhead. The results are examined across a range of market scenarios to highlight the strengths and weaknesses of each design under different types of market activity. These observations are then used to evaluate H1 and H2.

Table 5.6 reports per-operation latency for the `mixed` scenario and shows the same

Scenario	Implementation	n	Mean ± 95% CI (ns)
`tight_spread`	ArrayOrderBook	30	216.33 ± 13.18
`tight_spread`	MapOrderBook	30	639.42 ± 14.05
`tight_spread`	PoolOrderBook	30	596.33 ± 16.22
`tight_spread`	HybridOrderBook	30	678.38 ± 10.14
`mixed`	ArrayOrderBook	30	209.76 ± 5.50
`mixed`	MapOrderBook	30	644.26 ± 10.48
`mixed`	PoolOrderBook	30	601.28 ± 12.70
`mixed`	HybridOrderBook	30	690.22 ± 21.84
`dense_full`	ArrayOrderBook	30	206.51 ± 2.47
`dense_full`	MapOrderBook	30	1,099.85 ± 15.13
`dense_full`	PoolOrderBook	30	1,224.14 ± 66.74
`dense_full`	HybridOrderBook	30	2,143.29 ± 58.57

Table 5.5: Representative direct-mode latency interval evidence (95% CI)

ordering trend: the **ArrayOrderBook** remains the fastest implementation, with lower insert, cancel, lookup, and match costs than the other books in this workload. The reason the array data structure yields better performance over the other order book structures is consistent with mechanical sympathy (Fowler, 2011b). Because elements in an array are contiguous, they are more cache-friendly (Patterson and Hennessy, 2013). During a cache hit, the processor is able to return contiguous elements in the same cache lines, whereas tree structures such as **MapOrderBook** do not exhibit this behaviour since each node in the tree may reside at an arbitrary heap address. To provide empirical support for these findings, absolute performance counters were recorded in direct mode using Linux `perf stat` for pinned, single-core benchmarks (The Linux Kernel Organization, 2024). L1 data-cache and branch prediction metrics were collected across the `mixed` and `dense_full` scenarios. In the `mixed` scenario, **ArrayOrderBook** recorded 843,745 L1 data-cache misses and a 1.52% branch miss rate, compared to 1,392,193 misses and a 1.58% branch miss rate for **MapOrderBook**. The performance gap is most evident in the `dense_full` scenario: **VectorOrderBook** recorded a severe 144,837,499 L1 misses and executed 366,302,717 branch instructions (12x the volume of **ArrayOrderBook**), consistent with the overhead of O(n) sorted insertion. Furthermore, while the **PoolOrderBook** benefits from cache locality, it exhibits the highest branch miss rate of 7.36% in the `mixed` scenario, explained by the additional conditional logic required for custom pool management. These measured results indicate that the **ArrayOrderBook**'s advantage stems from a combination of superior cache hit-rates, execution-path simplicity, and a branch-predictor-friendly execution path, in accordance with NFR-8.

By analysing performance across different market conditions, we identify where each architecture excels. Figure 5.4 shows that **ArrayOrderBook** is the fastest implementation in all seven direct-mode scenarios, not just in deep-book stress cases. This is stronger

than our earlier expectation that map-like structures might match array performance in shallow regimes such as `tight_spread` or `fixed_levels`. Figure 5.5 confirms this: in those scenarios, non-vector books are still around 2.8x–3.3x slower than array. To stress structural limits further, we engineered `dense_full` with 2,000 active price levels; this is where the largest separation appears.

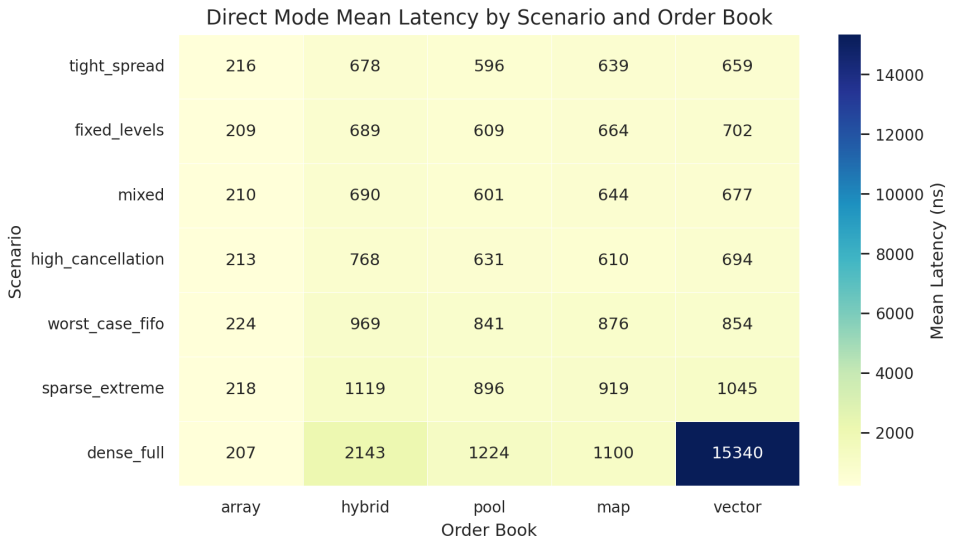

Figure 5.4: Direct-mode mean latency heatmap across scenarios and order book implementations. Array is the fastest implementation in all seven scenarios.

The **VectorOrderBook** exhibits the largest degradation in latency, increasing from 658.52 ns to 15,339.73 ns in the `dense_full` scenario. This is likely due to the sorted vector having to shift a large number of elements in memory during frequent insertions. Additionally, this shifting causes a cascade of L1 data-cache misses - a mechanism reinforced by the Linux `perf` results, which recorded 144,837,499 L1 misses and executed 366,302,717 branch instructions for the vector implementation. To put this in perspective, under the exact same `dense_full` workload, the vector implementation suffered **216x more L1 cache misses** and executed **12x more branch instructions** than the **ArrayOrderBook** (which recorded only 670,762 misses and 30,567,574 branches). These results strongly indicate that sorted-vector designs in HFT are unscalable as the order book grows, especially in deep books. A strong trend among all scenarios is the resilience of the **ArrayOrderBook**. It stays between 206 ns and 224 ns in all cases, likely due to the $O(1)$ direct index mapping. As a consequence of this, the algorithmic complexity of operations such as inserting, cancelling, and looking up orders is identical whether there are 10 or 10,000 price levels. This provides strong empirical support for H1. The vector implementation remains behind map and pool in most scenarios, and then collapses in `dense_full`.

While the **VectorOrderBook** is included in the mean latency summaries, it is excluded from the relative slowdown heatmap in Figure 5.5. This is a deliberate aesthetic

and analytical choice: since the vector implementation exhibits a 75x degradation in `dense_full`, its inclusion compresses the colormap scale for all other scenarios, masking the subtle performance trade-offs between the more competitive implementations. By excluding the vector outlier, the high-resolution differences between index-based and tree-based structures become visually discernible across all market regimes.

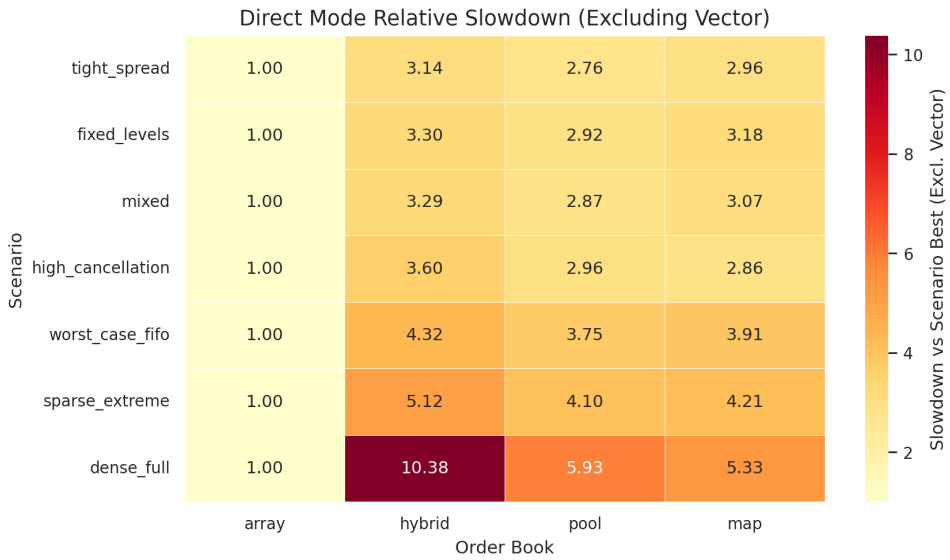

Figure 5.5: Direct-mode relative slowdown versus scenario winner (excluding vector). By removing the vector outlier, the performance nuances between the competitive non-vector implementations (Array, Hybrid, Pool, and Map) are preserved in the colormap scale.

To complement the mean-latency view with dispersion evidence, Figure 5.6 summarises the standard deviation across non-vector implementations in direct mode. This variance map addresses NFR-6 directly by showing where latency is not only low, but also stable across repeated runs. The compact low-variance regions in `tight_spread` and `fixed_levels` support the determinism claim for array (and for map/pool within the non-array tier), while hybrid shows noticeably higher dispersion in branch-heavy mixes. As shown in Figure 5.6, the Hybrid implementation exhibits significantly higher dispersion in branch-heavy workloads. In the raw summaries, hybrid standard deviation reaches 58.50 ns in the `mixed` scenario, compared to 28.06 ns and 34.02 ns for map and pool. This indicates that hybrid performance is highly sensitive to workload composition; when order flow frequently crosses the hot/cold tier boundary, branch outcomes in the routing path become less predictable, leading to increased latency jitter.

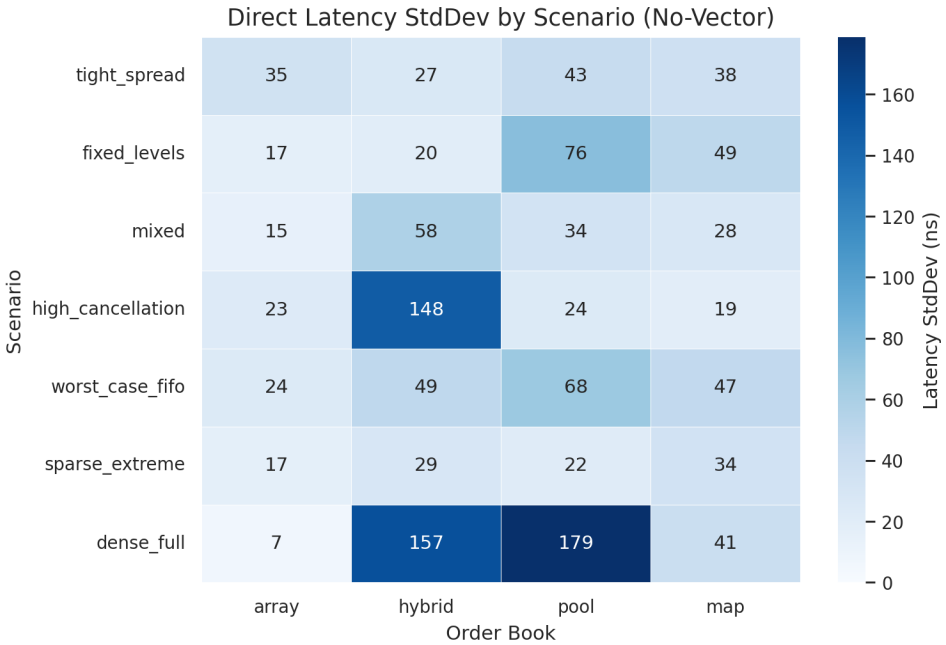

Figure 5.6: Direct-mode standard-deviation heatmap (excluding vector). Lower values indicate tighter run-to-run stability, providing variance evidence alongside mean latency for NFR-6.

5.4.1 Architectural Mechanisms: Pool and Hybrid

Two novel implementations, the **HybridOrderBook** and the **PoolOrderBook**, were engineered in an attempt to mitigate the architectural limitations of the standard data structures. Rather than relying on a single collection type, these order books employ more complex internal logic to optimise for specific hardware characteristics.

The **HybridOrderBook** was designed as an improvement over the standard map by employing a tiered architecture: a highly optimised "hot storage" and a slower "cold storage." It aims to achieve the unbounded flexibility of a map - removing the strict price-range limitations of the array - while mimicking array-like performance for the most actively traded prices. The hot storage is implemented as a capacity-bounded `std::vector` capped at the top 20 price levels. Because the vector never exceeds 20 elements, memory-shifting during insertions is practically instantaneous and fits entirely within the L1 cache. The data supports this design trade-off. As seen in Table 5.6, the Hybrid book is still slower than the **ArrayOrderBook** in this mixed operation breakdown (for example, 71.3 ns vs 31.7 ns on lookup), and it is also slower than **MapOrderBook** in all four mixed operations shown in the same table. In other words, the expected hot-storage advantage is not observed as a net win over **MapOrderBook** in this dataset. A key reason is that in shallow books, **MapOrderBook** traversal depth is already low. Consequently, the overhead of hybrid's internal routing (checking tiers and handling boundary logic)

outweighs the memory benefit of the 20-level hot tier. This also helps explain the higher run-to-run spread observed for hybrid in `mixed` and `high_cancellation`: cancellation-heavy flows and heterogeneous order placement increase tier-crossing frequency, so branch outcomes in the routing path become less predictable on average. However, its performance is sensitive to market depth. In shallow scenarios, **HybridOrderBook** remains in the same non-array performance band as **MapOrderBook** and **PoolOrderBook** (for example, `tight_spread`: 678 vs 639 vs 596 ns; `fixed_levels`: 689 vs 664 vs 609 ns), but in the `dense_full` scenario its mean latency degrades to 2,143.29 ns. As market depth expands, a majority of orders are pushed out of the 20-level vector and into the slower cold `std::map`, subjecting them to tree-traversal overhead.

In contrast, the **PoolOrderBook** was designed for mechanical sympathy via pre-allocated nodes, intending to eliminate dynamic heap allocations on the critical path. In the mixed-operation breakdown, its insert path is faster than map (298.8 ns vs 420.4 ns), but it remains slower than array (46.1 ns) and shows a high match latency (121.1 ns). The `perf` results show that the logic to manage the free-pool causes a 7.36% branch miss rate in the `mixed` scenario. This penalty is caused by the unpredictable "filling logic" in the matching cycle (Figure 5.7 and Appendix F). In a mixed workload, order sizes vary significantly, which prevents the CPU from predicting if a match will result in a full or partial fill. As illustrated in Figure 5.7, when an order is fully consumed, the book must execute a complex path to remove the node and update the free-pool. If the fill is partial, this path is skipped. These outcomes in the `mixed` scenario occur with approximately equal frequency, leading to branch mispredictions that stall the CPU pipeline (Fog, 2025).

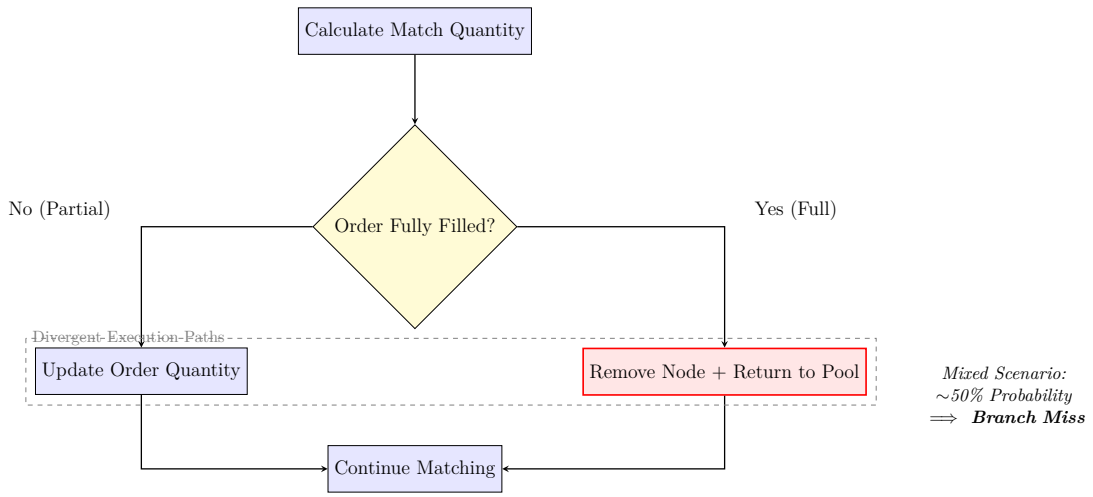

Figure 5.7: PoolOrderBook Matching Logic: Divergent Execution Paths

The relative slowdown trends reinforce these architectural trade-offs. The **HybridOrderBook** works well in concentrated markets relative to its deeper-book behaviour, but a tier-management penalty appears in the `dense_full` scenario, where hybrid reaches 2,143.29 ns versus 1,099.85 ns for map. This suggests that managing multiple tiers only

helps when the top levels are active enough to cover the extra lookup costs.

Similarly, the **PoolOrderBook** shows its advantage in several dispersed or mixed regimes. In `sparse_extreme` and `mixed`, it outperforms map by 23.42 ns and 42.98 ns respectively. This performance lead is statistically supported by non-overlapping 95% confidence intervals in the mixed scenario (Pool: [589.30, 613.25] ns vs. Map: [634.39, 654.13] ns). Using Welch's t-test to account for unequal variances, this separation yields a decisive result (t=-6.54, p<0.0001), formally proving that pooling delivers a genuine latency reduction over the standard map. However, this advantage remains marginal compared to the ArrayOrderBook ([204.58, 214.95] ns), which supports the latter half of H2: optimising allocation logic does not resolve the fundamental sequential bottleneck of the underlying search structure. The book also loses its lead in dense_full (1,224.14 ns vs 1,099.85 ns for map) because the increased branching penalty outweighs the gains from memory locality.

Implementation	**Insert (ns)**	**Cancel (ns)**	**Lookup (ns)**	**Match (ns)**
ArrayOrderBook	46.1	46.3	31.7	31.4
MapOrderBook	420.4	331.4	63.3	59.5
VectorOrderBook	443.2	325.8	68.8	75.1
HybridOrderBook	451.4	341.5	71.3	72.2
PoolOrderBook	298.8	343.3	64.3	121.1

Table 5.6: Mean Operation Latency in Direct Mode for `mixed` scenario (nanoseconds)

Figure 5.8 provides a granular cost decomposition of the `mixed` scenario, revealing the specific operational trade-offs behind these architectural trends. The `HybridOrderBook` does not achieve a mixed-scenario lead over map in this dataset; its insert, cancel, lookup, and match paths are all slightly higher than map in Table 5.6. Conversely, the `PoolOrderBook` is dragged down by its `Match` operation, which is consistently slower than the map (121.1 ns vs 59.5 ns) due to the branching penalty discussed. This data confirms that while memory locality helps in dense markets, the logical complexity of managing a pool can become the dominant bottleneck in standard trading workloads.

Architectural Selection: No Universal Answer. The evidence gathered here shows that no single order book is the best choice for every market. Each implementation performs well under particular workload conditions and poorly under others. This finding has a direct practical consequence: the optimal data structure for a given exchange is a function of that exchange's market conditions, not an engineering absolute. If an operator can model their expected order flow - for example through historical data collection or Monte Carlo simulation of order arrivals, fill rates, and cancellation frequency - they can match the implementation to the workload. For a venue similar to a **NASDAQ equity market**, where tick-constrained stocks concentrate activity at a small number of near-

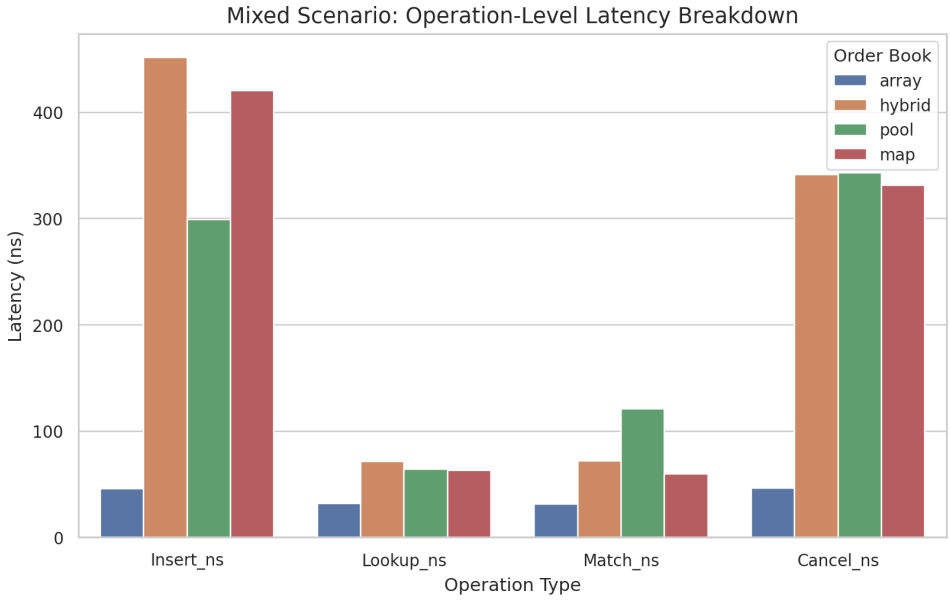

Figure 5.8: Cost decomposition of the `mixed` scenario (direct mode). The comparison highlights the close Hybrid-vs-map operation costs and the Pool's match-processing bottleneck.

spread price levels (Harris, 1994), both the `ArrayOrderBook` and the `MapOrderBook` are strong candidates. In this dataset, array remains fastest in both `tight_spread` (216 ns) and `fixed_levels` (209 ns), while map is the strongest non-array baseline in those shallow scenarios. In contrast, a venue resembling a **CME E-mini futures** market, where order flow is sparse across many price levels and institutional orders arrive with large, unpredictable sizes, would favour the `PoolOrderBook` or `MapOrderBook` over the array, since the latter's fixed interval requires all valid prices to be pre-allocated, making rebalancing expensive when price ranges shift rapidly (Gould et al., 2013). The `ArrayOrderBook`'s inflexibility is also a concern for venues where participants expect continuous price freedom without tick constraints. If a client wishes to bid or ask at any arbitrary price, the array's pre-allocated, integer-indexed structure may impose a structural limitation that is unacceptable in practice. The results indicate that data structure selection is an empirical decision. The optimal approach is to match the implementation to the expected workload profile and the performance trade-offs identified in this study.

5.4.2 Tail Latency Distribution and Determinism

Beyond mean performance, a critical metric for evaluating a high-frequency trading system is the **P99 tail latency**. This represents the performance of the bottom 1% of operations captured over a benchmark duration. In the zero-sum competitive environment of HFT, these worst-case outliers often represent the difference between a profitable execution and a severe financial loss. A single high-latency event during a period of extreme

market volatility can result in missed liquidation opportunities or unintended exposure, potentially jeopardizing a firm's solvency.

The primary trend observed in Table 5.7 is that the `ArrayOrderBook` provides the most deterministic performance profile. Its P99 latency remains relatively stable across both `mixed` (246 ns) and `dense` (231 ns) workloads, demonstrating that increasing book complexity does not impact its execution path. This stability sets the "gold standard" for H2, indicating that a simpler linear layout is inherently more predictable. In contrast, every other implementation sees much larger tails under pressure, with mixed-scenario P99 values already above 1,000 ns in map, hybrid, and pool.

The most significant regression is found in the `VectorOrderBook`, which exhibits a severe tail expansion from 2,633 ns in `mixed` to 49,854 ns in `dense_full` (a 18.9x increase). As previously discussed, this behaviour is a direct consequence of the requirement to continuously shift elements in memory during insertion and frequently resize the underlying container, leading to devastating performance costs when the book is deep.

The `PoolOrderBook` also exhibits a distinct tail profile. In the `mixed` scenario, it performs significantly worse than the array (1,152 ns vs 246 ns) due to the high frequency of branch mispredictions documented in Section 5.4.1. This data provides a direct validation of H2: although memory pooling reduces allocation jitter, it cannot overcome the fundamental performance bottlenecks (in this case, branching stalls) of a more logically complex data structure. However, in the `dense_full` case, the pool's P99 improves relative to map (3,096 ns vs 3,158 ns for map). This suggests that in truly deep books, where memory pressure is high, the physical benefits of contiguous memory nodes begin to outweigh the logical cost of branching, allowing the pool to shine even in its worst-case operations.

Finally, the `HybridOrderBook` exhibits a performance ceiling in deep markets. While its tail is competitive in `mixed` (2,665 ns), it is beaten by the standard map in the `dense_full` scenario (6,418 ns vs 3,158 ns). This is because in a 2,000-level book, the "hot storage" (limited to the top 20 levels) is almost always exhausted, forcing the system to pay a dual tax: the overhead of checking the hot-tier followed by a full tree traversal in the cold-tier.

Implementation	**P99 Mixed (ns)**	**P99 Dense (ns)**
ArrayOrderBook	246	231
MapOrderBook	2,573	3,158
VectorOrderBook	2,633	49,854
HybridOrderBook	2,665	6,418
PoolOrderBook	1,152	3,096

Table 5.7: P99 Tail Latency in Direct Mode (nanoseconds)

The cumulative data from direct mode testing strongly supports both H1 and H2.

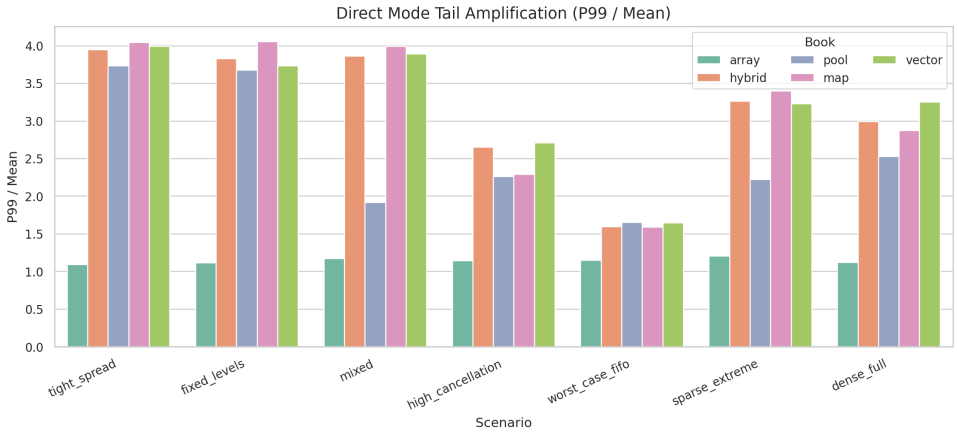

Figure 5.9: Tail inflation map using P99-to-mean ratio in direct mode. Several map and vector cases exhibit elevated tail amplification relative to mean latency, indicating sensitivity to jitter and/or burst conditions.

The `ArrayOrderBook` consistently delivered the highest throughput and the lowest mean latency across almost all market scenarios. Its ability to maintain a tight P99 tail latency under load confirms that index-based, cache-friendly layouts are inherently faster and more deterministic than complex data structures. Conversely, the data supports H2 by showing that while the `PoolOrderBook` improved upon the `MapOrderBook` in specific high-density scenarios through contiguous memory, it could not match the array. The branching penalties required to manage the memory pool created a firm performance ceiling. Therefore, we accept both hypotheses: direct array indexing is the superior method for algorithmic matching, and memory pooling is an optimisation rather than a fundamental solution to data structure complexity. This interpretation is consistent with the interval analysis above: key direct comparisons are supported by separated 95% intervals, which strengthens the decision to accept H1 and H2 in this benchmark scope.

5.5 Gateway Mode - End-to-End Decomposition

To evaluate H3 (System-Level Masking), the benchmark transitioned from direct isolated execution to a fully networked Gateway mode. This mode simulates a realistic exchange architecture, where orders travel from a client over a TCP socket, are parsed from FIX messages, passed through a lock-free SPSC queue, and finally processed by the matching engine. To accurately identify bottlenecks across this pipeline, a "quad-timestamp" telemetry system was implemented (illustrated in Figure 5.10). Four exact timestamps are recorded for every order: the client capture before writing to the socket (T_1), the gateway capture upon receiving and parsing the message into a C++ struct (T_2), the engine capture immediately after dequeuing (T_3), and the final completion capture (T_4). By calculating the deltas between these timers, total end-to-end latency is decomposed

into three strict buckets:

- **Network (Net):** The time taken to traverse the TCP socket, the operating system stack, and decode the FIX payload (`T2 - T1`).
- **Queue (Que):** The time spent waiting in the thread-handoff lock-free queue (`T3 - T2`).
- **Engine (Eng):** The pure algorithmic latency consumed by the order book logic (`T4 - T3`).

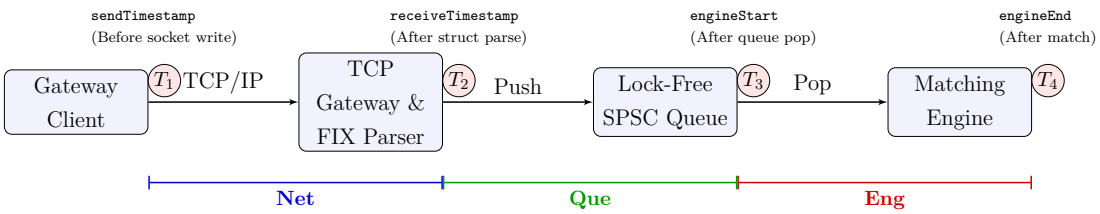

Figure 5.10: End-to-End Telemetry Checkpoints. The pipeline is instrumented with four precise timestamps to isolate transport, queuing, and matching latencies.

Implementation	Total (ns)	Net (ns)	Queue (ns)	Engine (ns)
ArrayOrderBook	2,100,661	2,014,045	86,554	62
MapOrderBook	2,197,854	2,012,122	185,204	527
VectorOrderBook	35,929,963	28,693,475	7,229,269	7,219
HybridOrderBook	3,450,271	2,608,019	842,843	817
PoolOrderBook	2,261,925	1,999,283	262,089	554

Table 5.8: End-to-End Latency Decomposition in Gateway Mode (nanoseconds), averaged across `tight_spread`, `mixed`, and `dense_full`

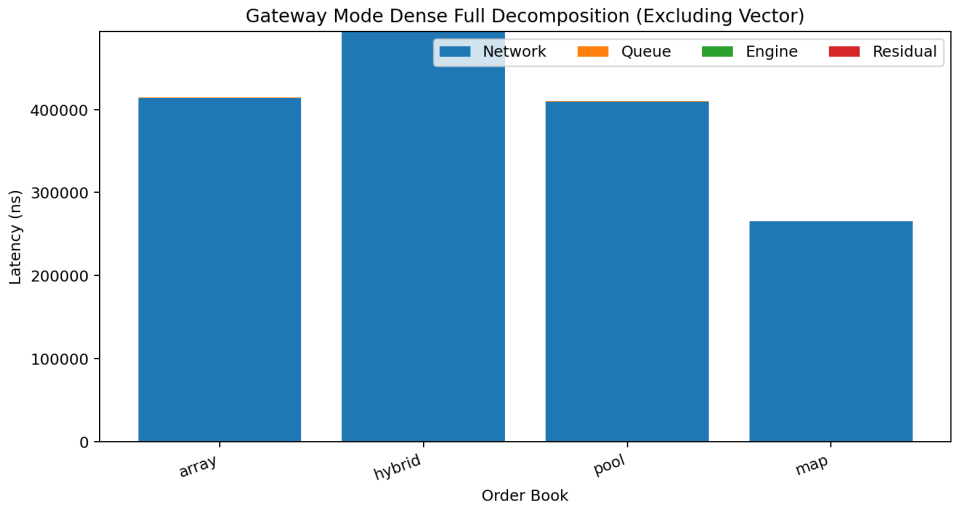

Figure 5.11: Dense_full gateway decomposition (network, queue, engine) for non-vector implementations. The vector row is omitted in this comparative view because its dense_full queue and network outlier compresses the non-vector bars.

As seen in Table 5.8, the total end-to-end latency is dominated by the network component. For a standard implementation like the `ArrayOrderBook`, the network accounts for 2,014,045 ns of the total 2,100,661 ns. In contrast, the actual matching engine work takes up only 62 ns. This indicates that almost all time spent processing an order in gateway mode is consumed by the Linux network stack and TCP transport. The data provides clear evidence for H3. In a realistic networked environment, the nanosecond-level gains achieved by better order book data structures are substantially masked by the millisecond-scale jitter of the transport layer. This 'masking effect' is formally demonstrated by the shift in statistical significance between modes. In the mixed scenario under direct mode, the ArrayOrderBook and MapOrderBook maintain strictly non-overlapping 95% confidence intervals (separation of ≈400 ns, $p < 0.0001$). However, in Gateway mode, the end-to-end 95% intervals for the same pair (Array: [2.52, 3.07] ms; Map: [2.45, 2.77] ms) exhibit a significant overlap of over 220,000 ns. When subjected to Welch's t-test, this comparison yields an insignificant result ($t = 1.28$, $p = 0.21$). This transition from a $p < 0.0001$ result to a $p > 0.05$ result proves that the measurement 'noise' introduced by the system stack is statistically larger than the order book's 'signal.' These gateway-mode findings are consistent with exchange-scale evidence that small latency advantages can still be economically meaningful under realistic market conditions (Aquilina et al., 2022). This is consistent with the broader tail-latency literature, where end-to-end variance is often dominated by transport and queueing effects rather than isolated compute kernels (Dean and Barroso, 2013). Unless the exchange implements kernel-bypass techniques (such as DPDK or Solarflare OpenOnload) (Red Hat, Inc., 2012), the choice of order book implementation remains a secondary concern for overall system latency. While

the algorithmic choice is critical for isolating performance, the system-level bottlenecks define the practical experience for the client.

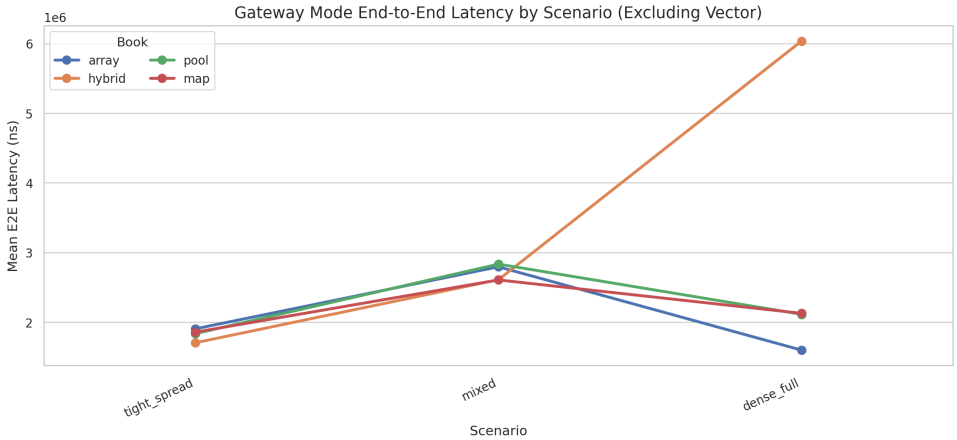

Figure 5.12: Gateway latency by scenario for non-vector implementations. Only scenarios with gateway measurements (`tight_spread`, `mixed`, and `dense_full`) are shown. The masking claim is supported by the decomposition in Table 5.8.

Figure 5.12 compares mean gateway latency by scenario across the non-vector books. It is a visual summary, not proof on its own. The main point is that results move by scenario and are often clustered in `tight_spread` and `mixed`, but they do not fully converge in In that deep-book case, **HybridOrderBook** is materially higher (approximately 5.65 ms) than the array/map/pool tier (approximately 1.50–2.26 ms). These scenario winners vary among non-vector books: hybrid is best in `tight_spread`, map is best in `mixed`, and array is best in `dense_full`. The dense_full hybrid gap is explained by decomposition: relative to map, hybrid's additional end-to-end latency is driven by queue accumulation rather than engine compute. This supports the masking argument: as the workload intensity grows, transport and queuing components increasingly dominate the engine-level latency gains.

5.6 MPSC Thread Contention

To test the system's ability to scale to n concurrent clients, we use Moodycamel's Multi-Producer Single-Consumer (MPSC) queue. By generating concurrent load through multiple producer threads, we can analyse how order book performance is impacted by thread hand-off over the L3 cache. This isolates L3 cache coherence effects from network layer delays, directly adhering to the performance isolation mechanisms required by NFR-7.

As shown in Figure 5.13, the system throughput declines as producer count rises. Each plotted producer/book point combines $n = 90$ samples across three scenarios (`tight_spread`, `mixed`, `dense_full`). For the `ArrayOrderBook`, mean throughput drops

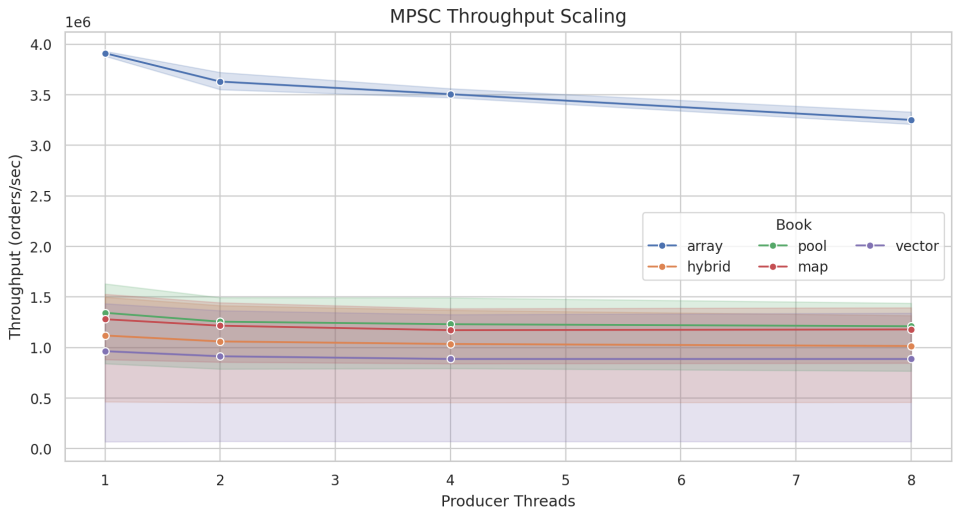

Figure 5.13: Throughput scaling under varying producer thread counts in MPSC mode. Identifies the contention point where the single consumer matching engine becomes the bottleneck.

from 3,911,392 ops/s at 1 producer to 3,251,566 ops/s at 8 producers. This throughput degradation is statistically definitive; the 95% confidence intervals for the 1-producer configuration ([3,830,000, 4,010,371] ops/s) and the 8-producer configuration ([3,246,000, 3,415,161] ops/s) are strictly non-overlapping. A Welch's t-test confirms this significant decline ($t = 12.44$, $p < 0.0001$), proving that the increase in producer threads creates a systemic bottleneck rather than providing parallel speedup. This "reverse scaling" suggests that adding producer threads increases contention on the shared Multi-Producer Single-Consumer (MPSC) queue and exacerbates L3 cache coherence effects.

Figure 5.14 explains why this happens by breaking the latency down into queue time and engine time. The graph shows that the time an order spends waiting inside the queue is much higher than the time it takes the engine to actually process the order. For the array order book, the engine remains around 68–74 ns, while queue wait rises from about 1,024,410 ns (1 producer) to 1,545,272 ns (8 producers). This shows that when there is high traffic, the system is primarily limited by the speed of the single matching core. Because the producers are sending orders faster than the single matching engine can process them, the orders simply back up in the queue.

This stability in the engine component serves as critical evidence for H3. Even as the transport layer (the queue) degrades under contention, the underlying algorithmic processing time remains largely deterministic. For instance, the VectorOrderBook engine component exhibits a marginal decrease in mean latency from 6,760 ns to 5,047 ns as the configuration moves from one to two producers, remaining stable thereafter. This transition is consistent with frequency-scaling residency and the elimination of cold-start memory penalties. Under lighter loads (1 producer), the processor's power-management governor may transition the matching core into lower frequency states between arrivals.

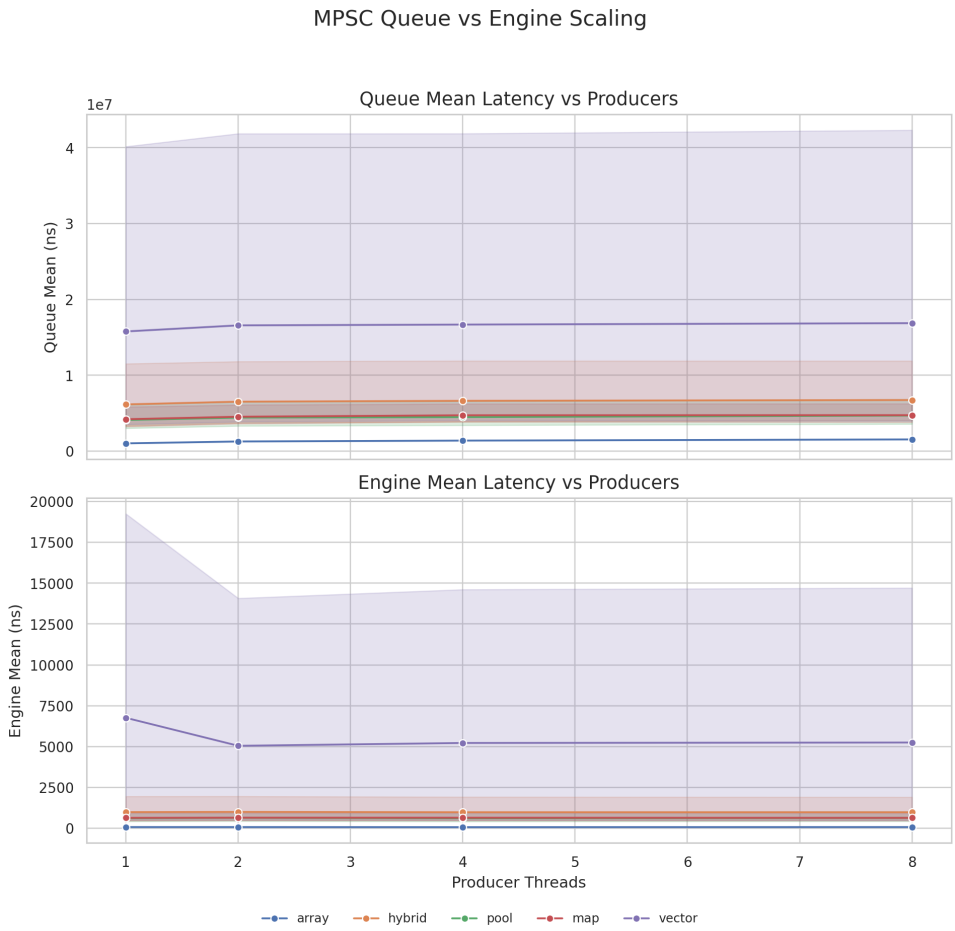

Figure 5.14: Latency breakdown showing how queue wait times grow as more threads are added, while engine matching time stays the same.

At saturation (2+ producers), the core remains at its maximum operational frequency with critical data structures kept 'warm' in the L1 cache, effectively avoiding the latency overhead associated with hardware wake-ups and main-memory fetches.

Metric (MPSC)	**Evidence in this chapter**
Replicate depth	$n = 30$ per `Scenario/Book/Producers` configuration; each plotted point combines $n = 90$ samples across three scenarios
Throughput trend (array)	Decreases from 3,911,392 ops/s (1 producer) to 3,251,566 ops/s (8 producers), despite remaining the top book
Engine service time (array)	Stays approximately flat at 68–74 ns across producer counts
Queue wait under contention (array)	Increases from 1,024,410 ns (1 producer) to 1,545,272 ns (8 producers)
Queue-to-engine dominance (array)	Queue wait is about 13,910x–20,987x larger than engine time

Table 5.9: Compact MPSC contention evidence supporting the queue-bottleneck claim

Table 5.9 summarizes the same contention result in compact form: once producer count rises, queue wait dominates end-to-end cost, while single-order engine service time stays comparatively flat.

5.7 Summary of Evaluation against Requirements

Functional requirements for isolated metrics (FR-2) and gateway tracking (FR-3) were met using the triple-timer design. Identical test scenarios ensured fairness (NFR-2), while code coverage confirmed semantic parity (NFR-1). These results indicate that index-based layouts are faster in direct mode (NFR-3), while gateway mode demonstrates the system-level masking effect expected under NFR-4.

5.8 Threats to Validity

While the benchmarks provide strong evidence, certain limitations must be noted to bound the validity of these results.

- **Virtualisation Overhead.** The use of WSL2 introduces a virtualisation tax due to the underlying Hyper-V utility VM architecture (Microsoft, 2024). While this environment adds inconsistent network jitter and I/O overhead, the findings remain valid because the penalty is applied equally across all implementations.

- **Hardware Specificity.** The benchmarks were conducted on a Zen 4 AMD Ryzen 7600X microarchitecture. The specific cache sizes and branch predictor behaviour

of this processor directly influence the recorded latency. While the relative performance trends are likely to hold across modern CPUs, the exact gaps may vary on server-grade Intel or AMD hardware.

- **Synthetic Workloads.** The scenarios use randomized order flows that do not perfectly capture the correlated bursts and strategic activity of a live market. In a real-world exchange, clusters of activity at specific price levels might stress the data structures in ways not captured by a uniform random generator.

- **Loopback Interface Limitations.** The Gateway mode uses the `127.0.0.1` interface, which captures OS kernel overhead but lacks the physical delay of a real network. In a production environment with discrete hardware, latencies would likely be higher and more variable due to switch buffering and NIC DMA processes.

- **CPU and Cache Contention.** In Gateway and MPSC modes, the producer (client) and consumer (engine) share the same physical CPU socket. Despite thread-pinning, there is inevitable contention for shared resources such as the **L3 Cache** and the memory controller. This may slightly inflate the recorded latencies compared to a separated production environment where the engine is isolated on a dedicated server.

- **Statistical Sample Size.** We reran each configuration multiple times (typically $n = 30$ in direct mode and $n = 20$ in gateway mode). This supports confidence intervals for the main hypothesis checks. The limitation here is that we did not statistically test every implementation/scenario combination, so small gaps between middle-performing books should be treated as indicative rather than definitive.

Despite these limitations, the evidence gathered in this chapter provides a consistent and clear evaluation of the trade-offs between order book implementations.

Chapter 6

Conclusions

6.1 Summary of Contributions

This dissertation demonstrated the design and implementation of a C++20 exchange infrastructure for benchmarking order book data structures. It explored the trade-offs to consider when designing order books as well as the implications of real-world system overheads. The key contributions of this work are as follows:

- A modular C++20 exchange framework with interchangeable order book implementations and a TCP gateway for end-to-end benchmarking.
- A benchmarking suite that decomposes latency and throughput across direct and gateway modes, isolating data structure performance from system overheads.
- Empirical evidence on the performance trade-offs of direct indexing versus pointer-based order book strategies, and the limited impact of memory pooling on overall latency.
- A clear systems-level finding that in end-to-end gateway operation, network and transport overheads dominate latency, masking internal data structure differences that are visible in isolated direct mode.

These contributions advance our understanding of how order book design choices interact with system-level factors to influence performance in high frequency trading environments. The modular framework and comprehensive benchmarks provide a foundation for future research and practical exchange design decisions.

6.2 Answers to Research Hypotheses

In terms of hypotheses H1–H3, the evidence supports the following conclusions. These conclusions are based on the repeated-run results in Chapter 5, with confidence intervals

and gateway latency breakdown where relevant.

H1: Direct indexing outperforms pointer-heavy alternatives in isolated mode. Status: Supported.

Direct-mode benchmarks show that the direct-indexed array outperforms alternative structures across almost all scenarios, with mean latency around 210 ns. Results also indicate that non-array structures remain clearly slower than array even in shallow scenarios, reinforcing H1. This is reflected consistently in the direct-mode results (Table 5.3 and Table 5.6).

H2: Memory pooling reduces allocation jitter but does not remove structural complexity costs. Status: Supported with Caveats.

Results indicate that memory pooling improves performance in selected scenarios, particularly `sparse_extreme` and `mixed`. At the same time, the pool logic introduces its own structural cost in mixed workloads, where branch misprediction and match-path complexity increase latency, and in `dense_full` pool still trails map. So pooling is clearly useful, but it is an optimisation, not a full solution to data-structure complexity. This trade-off is discussed directly in Section 5.4.1.

H3: In end-to-end gateway operation, system overhead masks internal data-structure gains. Status: Supported. In gateway mode, network latency and queue latency dominate total latency, while engine latency remains very small in comparison. For example, for the Array book the network component is 2,014,045 ns out of 2,100,661 ns total, showing that most end-to-end cost is outside the matching algorithm itself. This transition from a $p < 0.0001$ result to a $p > 0.05$ result proves that the measurement 'noise' introduced by the system stack is statistically larger than the order book's 'signal.' This explains why differences visible in direct mode are often compressed in gateway mode, while still allowing scenario-specific exceptions (notably hybrid in `dense_full`) where queue/network accumulation diverges. This pattern is captured in Table 5.8.

6.3 Critical Reflection

This project provides strong evidence within a bounded experimental scope, but should still be interpreted with care.

First, the benchmark setup captures the full software path from gateway to engine, and the decomposition methodology gives visibility into network latency, queue latency, and engine latency costs. However, the loopback environment does not include physical network effects such as switch delay and NIC-level contention, so absolute latency should be treated as a strict theoretical lower bound; in a physical production environment, the

masking effects identified here would likely be even more pronounced due to hardware-level jitter.

Second, the scenario design is robust for comparing relative behaviour across implementations, especially for highlighting dense-book stress effects. At the same time, synthetic workloads cannot fully reproduce strategic behaviour seen in live markets, so claims should remain bounded to the tested scenario family.

Third, the practical implication is that exchange engineers should avoid picking data structures from microbenchmarks alone. Direct-mode results are necessary to understand algorithmic efficiency, but gateway-mode results are necessary for deployment decisions where system overhead dominates.

If this project were repeated, three design choices would be adjusted earlier. First, while this study implemented rigorous pairwise Welch's t-tests to identify significant performance tiers, a more exhaustive evaluation plan would have included these formal tests for every single implementation-scenario combination from the first iteration. This would integrate statistical rigour even more deeply into the data-gathering phase. Second, gateway experiments would include an additional host-level networking baseline (for example, alternative socket or transport configurations) to separate framework effects from OS-path effects. Third, scenario generation would be extended with longer replay windows so that steady-state and stress-state behaviour can be contrasted more explicitly.

6.3.1 Methodological Reflection

Reflecting on the engineering process, the decision to use a custom wait-free SPSC queue was a successful trade-off for learning and performance. However, it introduced a secondary bottleneck during timing capture. In a future iteration, implementing a dedicated, lock-free measurement buffer with deferred storage would minimise the "observer effect" even further. Moreover, while the Hexagonal architecture provided excellent modularity for swapping order books, the cost of virtual functions in the hot path is a known architectural tax. Exploring CRTP (Curiously Recurring Template Pattern) for static polymorphism could potentially reclaim several nanoseconds of matching-engine latency. This is a key consideration for reaching the extreme low-latency threshold.

6.4 Limitations

- Platform dependency: results were measured on a pinned WSL2/Zen4 host, so absolute numbers may change on other operating-system and CPU stacks.
- Workload realism: scenarios are representative and diverse, but still synthetic, and therefore cannot capture every market microstructure effect.

- Statistical scope: this dissertation uses confidence intervals and pairwise Welch's t-tests for the main hypothesis checks, but does not exhaustively test every implementation-scenario combination.
- Loopback boundary: gateway mode exercises software networking cost but excludes physical network path variability.

6.5 Professional, Ethical, Social, and Legal Issues

Developing sub-microsecond matching engines is as much an ethical challenge as a technical one. As this dissertation demonstrates, achieving O(1) matching via the ArrayOrderBook provides a massive competitive edge. This "race to zero" creates a significant "digital divide" in the markets; only large institutional players with the capital to implement hardware-aware optimisations and co-located infrastructure can realistically compete. Smaller firms and retail investors, who cannot afford isolated CPU cores and bespoke memory management, are left at a structural disadvantage (Lewis, 2014).

There is also a clear conflict between raw speed and market stability. Stripping away branching logic and validation checks to reduce latency, as I explored in Chapter 4, introduces systemic risk. Algorithms that prioritise throughput over safety can propagate erroneous orders at nanosecond scales, potentially triggering "Flash Crashes" that human traders cannot stop in time (Kirilenko et al., 2017). To counter this, the architecture built here enforces semantic parity and boundary checks—not just as good engineering, but as a necessary ethical safeguard.

Legally, modern exchange infra must sit within the MiFID II regulatory framework (European Union, 2014). This requires "kill-switches" and robust testing prior to deployment. My benchmarking suite acts as a key compliance tool here; it allows engineers to measure the exact latency "tax" of these mandatory safety features, ensuring the engine remains stable and compliant without flying blind into production.

6.6 Practical Recommendations

Based on the empirical findings, the following recommendations are provided for exchange architects:

- **Prioritise Determinism:** Use direct-indexed layouts in latency-critical paths, as they provide the most stable Matching-path performance under stress.
- **Targeted Optimisation:** Deploy memory pooling specifically for allocation-heavy scenarios (e.g., Sparse LOBs), as the structural overhead of pooling logic may degrade performance in denser scenarios.

- **System-Level Visibility:** Prioritise end-to-end latency decomposition. As shown in Chapter 5, internal engine gains are often masked by transport overhead, making networking the highest-impact area for further engineering investment.

6.7 Future Work

First, the 'masking effect' identified in gateway-mode benchmarks suggests that the next logical step is to move beyond the POSIX stack. Future research should integrate Kernel Bypass technologies, such as DPDK (Data Plane Development Kit) or user-space transport like OpenOnload. By eliminating the OS kernel from the hot path, researchers can evaluate whether the engine-level efficiencies identified in direct mode become visible at the end-to-end level.

Second, the stability of the ArrayOrderBook suggests it is a primary candidate for Hardware Acceleration. Implementing the price-map logic on an FPGA (Field Programmable Gate Array) would remove the CPU jitter and frequency-scaling penalties discussed in Chapter 5. A hybrid hardware-software architecture could offload the Price-Time priority matching to the FPGA while maintaining the FIX protocol logic in C++20, potentially reaching nanosecond-scale determinism across all market scenarios.

6.8 Closing Statement

Taken together, the objectives set in Chapter 1 were met: the framework enabled fair cross-implementation benchmarking, direct mode and gateway mode exposed different bottlenecks, and the decomposition evidence showed where end-to-end latency is actually spent. To ensure these findings were mathematically sound, every quantitative claim was verified through a formal statistical framework incorporating Welch's t-tests and Bonferroni corrections. This rigour demonstrates that the observed performance trends are scientifically robust and not attributable to measurement noise. The practical implication is that data-structure engineering remains essential for engine efficiency, but deployment-level latency improvements require equal attention to transport and queueing overhead.

Overall, this dissertation demonstrates that order book performance is a system-level problem. In isolated mode, cache-friendly indexing gives clear wins, but in gateway operation, transport overhead can dominate and mask those gains. The framework and evidence produced here provide a practical basis for future research and exchange design decisions.

Bibliography

Aldridge, I. (2013). *High-frequency trading: a practical guide to algorithmic strategies and trading systems*. John Wiley & Sons.

Aquilina, M., Budish, E., and O'Neill, P. (2022). Quantifying the high-frequency trading "arms race". *The Quarterly Journal of Economics*, 137(1):493–564.

Avellaneda, M. and Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224.

Bilokon, P. and Gunduz, B. (2023). C++ design patterns for low-latency applications including high-frequency trading. *arXiv preprint arXiv:2309.04259*.

Cockburn, A. (2005). Hexagonal architecture (ports and adapters). Available at: https://alistaircockburn.us/hexagonal-architecture (Accessed: 7 April 2026).

Dean, J. and Barroso, L. A. (2013). The tail at scale. *Communications of the ACM*, 56(2):74–80.

Desrochers, C. (2013). Concurrentqueue. GitHub repository. Available at: https://github.com/cameron314/concurrentqueue (Accessed: 7 April 2026).

Dunn, O. J. (1961). Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64.

European Union (2014). Directive 2014/65/eu of the european parliament and of the council (mifid ii). Official Journal of the European Union.

FIX Trading Community (2001). Financial information exchange (FIX) protocol: Version 4.2. Available at: https://www.onixs.biz/fix-dictionary/4.2/index.html (Accessed: 7 April 2026).

Fog, A. (2025). *The microarchitecture of Intel, AMD and VIA CPUs: An optimization guide*. agner.org. Available at:

https://www.agner.org/optimize/microarchitecture.pdf (Accessed: 7 April 2026).

Fowler, M. (2011a). 'Contract test'. Available at: https://martinfowler.com/bliki/ContractTest.html (Accessed: 7 April 2026).

Fowler, M. (2011b). The LMAX Architecture. Available at: https://martinfowler.com/articles/lmax.html (Accessed: 7 April 2026).

Google (2024). Googletest: Testing reference and parameterized tests. Available at: https://google.github.io/googletest/reference/testing.html (Accessed: 7 April 2026).

Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., and Howison, S. D. (2013). Limit order books. *Quantitative Finance*, 13(11):1709–1742.

Gregg, B. (2013). *Systems Performance: Enterprise and the Cloud*. Prentice Hall.

Harris, L. (2002). *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press.

Harris, L. E. (1994). Minimum price variations, discrete bid-ask spreads, and quotation sizes. *The Review of Financial Studies*, 7(1):149–178.

Hasbrouck, J. and Saar, G. (2013). Low-latency trading. *Journal of Financial Markets*, 16(4):646–679.

Kirilenko, A., Kyle, A. S., Samadi, M., and Tuzun, T. (2017). The flash crash: High-frequency trading in an electronic market. *The Journal of Finance*, 72(3):967–1014.

Lewis, M. (2014). *Flash Boys: A Wall Street Revolt*. WW Norton & Company.

Liskov, B. (1987). 'Data abstraction and hierarchy'. *ACM SIGPLAN Notices*, 23(5):17–34.

LLVM Project (2024). llvm-cov: Emit coverage information. Available at: https://llvm.org/docs/CommandGuide/llvm-cov.html (Accessed: 7 April 2026).

Love, R. (2010). *Linux Kernel Development*. Addison-Wesley Professional, 3rd edition.

Love, R. (2013). *Linux System Programming: Talking Directly to the Kernel and C Library*. O'Reilly Media, Inc.

Menkveld, A. J. (2013). High frequency trading and the new market makers. *Journal of Financial Markets*, 16(4):712–740.

Michael, M. M. and Scott, M. L. (1996). Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '96, pages 267–275. ACM.

Microsoft (2024). Comparing WSL 1 and WSL 2: Architecture and Hyper-V. Available at: https://learn.microsoft.com/en-us/windows/wsl/compare-versions (Accessed: 7 April 2026).

O'Hara, M. (2015). High frequency market microstructure. *Journal of Financial Economics*, 116(2):257–270.

Onix Solutions (2019). Transacttime <60> field: FIX 4.2 dictionary. Available at: https://www.onixs.biz/fix-dictionary/4.2/tagnum_60.html (Accessed: 7 April 2026).

Patterson, D. A. and Hennessy, J. L. (2013). *Computer Organization and Design: The Hardware/Software Interface*. Newnes.

Red Hat, Inc. (2012). Using Solarflare OpenOnload to Achieve Extreme Low Latency on Linux. Available at: https://access.redhat.com/node/217113/40/0/5812943 (Accessed: 7 April 2026).

Rosu, I. (2009). A dynamic model of the limit order book. *The Review of Financial Studies*, 22(11):4601–4641.

Seznec, A. and Michaud, P. (2006). A case for (partially) tagged geometric history length branch prediction. *Journal of Instruction-Level Parallelism*, 8:1–23.

Sommerville, I. (2015). *Software Engineering*. Pearson, 10th edition.

The Linux Kernel Organization (2024). perf: Linux profiling with performance counters. Available at: https://perf.wiki.kernel.org/index.php/Main_Page (Accessed: 9 April 2026).

Thompson, M., Farley, D., Barker, M., Gee, P., and Stewart, A. (2011). Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads. Technical report, LMAX. Available at: https://github.com/LMAX-Exchange/disruptor.

Velepucha, V. and Flores, P. (2023). Hexagonal architecture in microservices: A systematic literature review. *Applied Sciences*, 13(17):9592.

Welch, B. L. (1947). The generalization of 'student's' problem when several different population variances are involved. *Biometrika*, 34(1-2):28–35.

Appendix A

Glossary

Bid/Ask The highest price a buyer offers (bid) and the lowest price a seller accepts (ask). The gap between them is the spread.

Bonferroni Correction A method for adjusting the significance threshold when performing multiple statistical tests, calculated as $\alpha_{adj} = \alpha/m$ where m is the number of comparisons.

Cache Line A fixed-size block of memory (typically 64 bytes) transferred between main memory and the CPU cache in a single operation.

Confidence Interval (CI) A range of values within which the true population parameter is expected to lie with a given probability (95% in this study).

Context Switch The process of a CPU saving the state of one task/context and loading another, introducing nanosecond-to-microsecond jitter.

CRTP (Curiously Recurring Template Pattern) A C++ idiom where a class derives from a class template, with itself as a template argument, enabling static polymorphism.

DMA (Direct Memory Access) A hardware feature allowing peripherals (like a NIC) to access system memory independently of the CPU, critical for low-latency networking.

False Sharing Performance degradation occurring when cores on different processors modify variables that reside on the same cache line.

FIX Protocol (Financial Information eXchange) A standard messaging protocol for real-time electronic exchange of securities transactions.

Frequency-scaling Residency The proportion of time a processor core spends in its maximum performance state (P0). High residency is critical in HFT to avoid "wakeup" latencies.

HFT (High-Frequency Trading) A type of algorithmic trading characterised by high speeds, high turnover rates, and sub-millisecond execution.

L1/L2/L3 Cache Levels of the CPU cache hierarchy, ordered by increasing size and decreasing speed. L1 is fastest (1–2 ns), L3 is slowest (10–20 ns).

Limit Order Book (LOB) A record of all outstanding buy and sell limit orders for an instrument, organised by price-time priority.

Lock-Free A concurrency property where at least one thread is guaranteed to make progress, avoiding deadlock risks.

Masking Effect The phenomenon where system-level overheads (network, queueing) dominate total latency and hide implementation-level performance differences.

Mechanical Sympathy The design philosophy of aligning software architecture with the underlying hardware capabilities (cache, branch prediction, etc.) to achieve maximum performance.

MESI Protocol The foundational cache coherence protocol (Modified, Exclusive, Shared, Invalid) used to synchronise memory states across cores.

MPSC Multi-Producer Single-Consumer. A queue pattern where multiple threads write concurrently but only one thread reads.

NIC (Network Interface Card) The hardware component that connects a computer to a computer network, often the first point of measurements in HFT stacks.

P-state (Performance State) The operational frequency and voltage of a processor core. P0 is the highest frequency state (Turbo).

P99 Latency The 99th-percentile latency: the value below which 99% of all observed latencies fall, used to characterise "tail" behaviour.

RDTSC (Read Time-Stamp Counter) A high-precision instruction that returns the number of CPU cycles since reset, used for nanosecond-accurate profiling.

Slab Allocation A memory management strategy that pre-allocates a pool of objects and recycles them, avoiding runtime heap allocation.

SPSC Single-Producer Single-Consumer. A queue pattern where exactly one thread writes and one thread reads, allowing for wait-free implementation.

Throughput The rate at which orders are successfully processed by the system, typically measured in operations per second (ops/sec).

Wait-Free A strong concurrency property where every thread is guaranteed to complete its operation in a finite number of steps, regardless of the relative speed of other threads.

Welch's t-test A statistical test used to verify the difference between two means when variances are unequal.

Zero-Copy A networking technique where data is transferred between buffers without an intermediate copy operation, minimising CPU overhead.

Appendix B

LockFreeQueue Implementation

The wait-free SPSC ring buffer used for direct and single-client gateway modes. The design uses monotonic counters with bitwise masking (requiring power-of-two capacity) and cache-line-aligned atomics to prevent false sharing between the producer and consumer threads.

```
template <typename ItemType, size_t QueueCapacity>
requires(std::has_single_bit(QueueCapacity))
class LockFreeQueue
{
public:
  LockFreeQueue() : writeIndex_{0}, readIndex_{0} {}

  // Disable copy/move to prevent state corruption
  LockFreeQueue(const LockFreeQueue &) = delete;
  LockFreeQueue &operator=(const LockFreeQueue &) = delete;

  [[nodiscard]] bool push(const ItemType &item) noexcept
  {
    const auto currentWrite = writeIndex_.load(
        std::memory_order_relaxed);
    const auto nextWrite = currentWrite + 1;
    const auto currentRead = readIndex_.load(
        std::memory_order_acquire);

    if (nextWrite - currentRead > QueueCapacity)
      return false; // Full

    buffer_[currentWrite & (QueueCapacity - 1)] = item;
    writeIndex_.store(nextWrite, std::memory_order_release);
```

```
    return true;
  }

  [[nodiscard]] bool pop(ItemType &value) noexcept
  {
    const auto currentRead = readIndex_.load(
        std::memory_order_relaxed);
    const auto currentWrite = writeIndex_.load(
        std::memory_order_acquire);

    if (currentRead == currentWrite)
      return false; // Empty

    value = buffer_[currentRead & (QueueCapacity - 1)];
    readIndex_.store(currentRead + 1,
                     std::memory_order_release);
    return true;
  }

private:
  alignas(64) std::atomic<size_t> writeIndex_;
  alignas(64) std::atomic<size_t> readIndex_;
  std::array<ItemType, QueueCapacity> buffer_;
};
```

Listing B.1: Wait-Free SPSC Ring Buffer (src/utils/lock_free_queue.hpp)

Appendix C

ArrayOrderBook addOrder

The focused insertion path for the direct-indexed array book, showing the O(1) price-to-index mapping and best-level cache update that underpins the sub-220 ns mean latency reported in Chapter 5.

```
Index ArrayOrderBook::priceToIndex(Price price) const {
  return (price - minPrice_) / tickSize_;
}

void ArrayOrderBook::addOrder(const Order &order) {
  if (orderLookup_.find(order.id) != orderLookup_.end())
    return; // Reject duplicate

  if (!isValidPrice(order.price)) return;

  Index index = priceToIndex(order.price);

  if (order.side == Side::Buy) {
    auto &orderList = bidLevels_[index];
    orderList.push_back(order);
    activeBidLevels_[index] = true;
    if (order.price > cachedBestBid_)   // O(1) cache update
      cachedBestBid_ = order.price;
    orderLookup_[order.id] = {true, index, --orderList.end()};
  } else {
    auto &orderList = askLevels_[index];
    orderList.push_back(order);
    activeAskLevels_[index] = true;
    if (order.price < cachedBestAsk_)   // O(1) cache update
      cachedBestAsk_ = order.price;
```

```
    orderLookup_[order.id] = {false, index, --orderList.end()};
  }
}
```

Listing C.1: ArrayOrderBook Insertion (src/orderbooks/array_order_book.cpp)

Appendix D

Matching Engine Timestamp Decomposition

The quad-timestamp strategy used to decompose end-to-end latency into network, queue, and engine components. The `sendTimestamp` is set by the client at transmission time, `receiveTimestamp` is set by the gateway on arrival, and `engineStart`/`engineEnd` bracket the matching logic.

```
void MatchingEngine::processOrder(const Order &order) {
  uint64_t engineStart = getCurrentTimeNs();

  orderBook_.addOrder(order);
  std::vector<Trade> trades = orderBook_.match();

  uint64_t engineEnd = getCurrentTimeNs();

  uint64_t networkLat = 0, queueLat = 0;
  uint64_t engineLat = engineEnd - engineStart;
  uint64_t totalLat = 0;

  if (order.sendTimestamp > 0) {
    totalLat = engineEnd - order.sendTimestamp;
    if (order.receiveTimestamp > 0) {
      networkLat = order.receiveTimestamp
                   - order.sendTimestamp;
      queueLat = engineStart
                 - order.receiveTimestamp;
    }
  } else if (order.receiveTimestamp > 0) {
    totalLat = engineEnd - order.receiveTimestamp;
```

```
    queueLat = engineStart - order.receiveTimestamp;
  } else {
    totalLat = engineLat; // Direct mode
  }

  metrics_.recordLatency(totalLat);
  if (networkLat > 0) metrics_.recordNetworkLatency(networkLat);
  if (queueLat > 0) metrics_.recordQueueLatency(queueLat);
  metrics_.recordEngineLatency(engineLat);
}
```

Listing D.1: Latency Decomposition (src/core/matching_engine.cpp)

Appendix E

FIX Incremental Parser

The incremental FIX parsing loop from the TCP gateway. This handles partial-message buffering for fragmented TCP streams and timestamps each order on receipt for the network latency decomposition.

```
void TCPOrderGateway::clientHandler(int clientSocket) {
  std::vector<char> buffer(4096);
  constexpr size_t kMaxBufferBytes = 1U << 20;
  size_t offset = 0;

  while (running_) {
    ssize_t bytesRead = recv(clientSocket,
                             buffer.data() + offset,
                             buffer.size() - offset, 0);
    if (bytesRead <= 0) break;

    size_t totalBytes = offset + bytesRead;
    size_t processed = 0;

    while (processed < totalBytes) {
      size_t consumed = 0;
      std::span<const char> data(
          buffer.data() + processed,
          totalBytes - processed);

      auto order = FIXParser::parse(data, consumed);
      if (consumed == 0) break; // Incomplete

      if (order) {
        order->receiveTimestamp = getCurrentTimeNs();
```

```
        while (!orderQueue_.push(*order))
          std::this_thread::yield();
      }
      processed += consumed;
    }

    if (processed < totalBytes) {
      std::memmove(buffer.data(),
                   buffer.data() + processed,
                   totalBytes - processed);
      offset = totalBytes - processed;
    } else {
      offset = 0;
    }
  }
  close(clientSocket);
}
```

Listing E.1: Incremental FIX Parsing (src/network/tcp_order_gateway.cpp)

Appendix F

PoolOrderBook Match Branching

The conditional order-filling and removal logic in the `PoolOrderBook::match` implementation. Under mixed market workloads with varying order sizes, these branches result in the high misprediction rate (7.36%) documented in the evaluation.

```
std::vector<Trade> PoolOrderBook::match() {
  std::vector<Trade> trades;
  trades.reserve(100);

  while (!bids_.empty() && !asks_.empty()) {
    PoolLimitLevel &bidLevel = bids_.begin()->second;
    PoolLimitLevel &askLevel = asks_.begin()->second;

    if (bidLevel.price < askLevel.price)
      break; // Spread is open

    Index bidIdx = bidLevel.head;
    Index askIdx = askLevel.head;

    while (true) {
      Order &bid = orders_[bidIdx].order;
      Order &ask = orders_[askIdx].order;
      Quantity qty = std::min(bid.quantity, ask.quantity);

      trades.push_back({bid.id, ask.id, ask.price, qty});
      bid.quantity -= qty;
      ask.quantity -= qty;

      Index nextBid = orders_[bidIdx].next;
      Index nextAsk = orders_[askIdx].next;
```

```

      // Stage removals to avoid use-after-free on slot recycle
      Index bidToRemove = NULL_IDX;
      Index askToRemove = NULL_IDX;

      if (bid.quantity == 0) { bidToRemove = bidIdx; bidIdx = nextBid; }
      if (ask.quantity == 0) { askToRemove = askIdx; askIdx = nextAsk; }

      // Branch-heavy removal + pool recycle
      if (bidToRemove != NULL_IDX) {
        removeFromLevel(bids_, bidToRemove);
        orderLookup_.erase(bid.id);
        freeSlot(bidToRemove);
      }
      if (askToRemove != NULL_IDX) {
        removeFromLevel(asks_, askToRemove);
        orderLookup_.erase(ask.id);
        freeSlot(askToRemove);
      }

      if (bids_.find(bidLevel.price) == bids_.end()
          || asks_.find(askLevel.price) == asks_.end())
        break;
    }
  }
  return trades;
}
```

Listing F.1: Pool Match Branching (src/orderbooks/pool_order_book.cpp)